High-Performance Computer Architectures Practical Course

Script

High-Performance Computing
Prof. Dr. Ivan Kisel
Robin Lakos
Akhil Mithran
Oddharak Tyagi

June 25, 2025

Dear students,

Welcome to the High-Performance Computer Architectures practical course. The goal of this course is to provide you an introduction to software development for high-performance computers including simplified applications from our daily research routine. The modern low-level programming language C++ allows to develop software close to the hardware specifications and allows to program runtime- and memory-efficient applications, running on CPU and GPU. Within this course, you will learn how to work with Unix-based systems and C++ to write smaller applications that make use of several concepts of parallel programming such as multi-threading and vectorization.

If you have any questions regarding the examples given, please ask your tutor.

Please note: During the course, we will add new sections and provide more examples for the specific topics to provide a good overview of the course requirements. If you have any suggestions or wishes that could help you, please let us know.

Contents

Chapter 1

Introduction - Unix Shell and C++

The Unix Shell and C++ are essential tools for anyone working in high-performance computing or scientific research. Both provide robust capabilities for managing and executing tasks efficiently. This chapter is divided into two sections: the first introduces the Unix Shell as a versatile command-line interface for interacting with operating systems, and the second provides an introduction to C++ programming, emphasizing its role in computational science.

1.1 Unix Shell

The Unix Shell acts as a powerful interface between users and the operating system, allowing direct interaction through text-based commands. Its flexibility and precision make it an indispensable tool for professionals, particularly in high-performance computing, where efficiency and automation are crucial. Unlike graphical user interfaces (GUIs), which prioritize user-friendliness, the shell excels in handling complex, repetitive, and large-scale tasks with ease, thanks to its focus on command-line execution and scripting.

Unix, developed in 1969, laid the foundation for many modern operating systems, including Linux and macOS. Its principles of multi-tasking, multi-user support, and portability have made it a cornerstone of computing systems worldwide. Today, Linux, a Unix-based kernel, powers a vast array of devices, from personal computers to high-performance computing (HPC) clusters and servers.

Linux's open-source nature is one of its greatest strengths ind HPC and research. It allows developers and researchers to inspect, modify, and optimize the source code to suit specific needs. This flexibility, coupled with contributions from a global community, ensures robust performance, rapid issue resolution, and high adaptability for demanding computational tasks.

Linux distributions come in many flavors, tailored to different use cases. For instance, Debian and Fedora are often chosen for servers due to their stability and reliability. Conversely, distributions like Ubuntu and Arch Linux cater to desktop users with user-friendly interfaces and up-to-date software. The availability of diverse Desktop Environments (DEs) further allows users to customize their workflows, enhancing productivity and accessibility.

In the realm of HPC, Linux dominates as the operating system of choice. Its stability, scalability,

and extensive support for open-source tools and frameworks make it indispensable for running large-scale simulations, managing big data, and optimizing computational workflows.

1.1.1 Terminal and Shell Commands

In this course, all C++ applications will be compiled and executed via the terminal (also referred to as the console), using the Unix Shell. Working through the terminal allows precise control and avoids the overhead introduced by graphical user interfaces or additional software. While initially it may feel less intuitive, the terminal is a powerful tool for efficient computing, particularly in high-performance computing environments.

When you start a terminal in Linux, the prompt typically displays `username@hostname`, followed by a *path*, which represents the current *working directory*. By default, the working directory is usually set to the user's home directory. Note that terminal prompts and styles may vary depending on the application or theme being used.

Key Commands for Navigation
The following table summarizes essential commands for navigating the file system in the Unix Shell:

Command	Description
`pwd`	Prints the working directory, showing the absolute path to the current location.
`cd`	Changes directory. Paths can be absolute (starting from the root directory, `/`) or relative (from the current directory).
`ls`	Lists the contents of the current directory.

Table 1.1: Essential Unix Shell commands for navigation.

Below is an example of using these commands in a typical workflow:

```
username@hostname:~$ pwd
/home/hpc
username@hostname:~$ ls
directory-A directory-B directory-C
username@hostname:~$ cd directory-A
username@hostname:~/directory-A$ ls
directory-1 directory-2
username@hostname:~/directory-A$ cd directory-1/directory-1-1
username@hostname:~/directory-A/directory-1/directory-1-1$ cd ../../..
username@hostname:~$
```

Switching between directories with long paths can become tedious. The command **cd -** offers a convenient shortcut, allowing you to quickly switch back to the last directory you were working in. This is especially helpful when toggling between two separate locations:

```
username@hostname:~$ pwd
/home/hpc
username@hostname:~$ cd directoryA/directory-1/directory-1-1
username@hostname:~/directoryA/directory-1/directory-1-1$ DO SOMETHING HERE
username@hostname:~/directoryA/directory-1/directory-1-1$ cd
↪  ../../../directoryB/directory-3/directory-2-1
username@hostname:~/directoryB/directory-3/directory-2-1$ DO SOMETHING THERE
username@hostname:~/directoryB/directory-3/directory-2-1$ cd -
username@hostname:~/directoryA/directory-1/directory-1-1$ DO SOMETHING HERE
↪  AGAIN
```

Mastering the shell is essential for anyone aiming to work efficiently in high-performance computing. Throughout this course, make an effort to familiarize yourself with the terminal, as it will be a key tool for compiling, running, and managing your C++ applications.

1.2 C++ Basics

C++ is one of the most widely used programming languages in scientific computing and high-performance applications. Known for its efficiency, flexibility, and extensive control over system resources, C++ combines the power of low-level hardware access with modern high-level abstractions. These features make it an indispensable tool in fields such as computational physics, engineering simulations, and data-intensive research. In this section, we introduce the foundational concepts of C++ and its relevance to modern programming.

1.2.1 The History and Evolution of C++

C++ was first developed by Bjarne Stroustrup in 1985 as an extension of the C language. The goal was to combine the performance and portability of C with features supporting object-oriented programming, such as classes and objects. This key enhancement distinguished C++ as one of the first mainstream programming languages to support object-oriented design, alongside procedural and generic programming paradigms. C++ has grown significantly since its inception, evolving through multiple standards established by the International Organization for Standardization (ISO). Starting from C++98, these standards have introduced features like templates, smart pointers, and lambda expressions, with major updates occurring every three years since 2011. This makes C++ a ”modern” language that balances its low-level control with high-level abstractions. While C++ is often considered a high-level language due to its expressive syntax, it retains low-level features like direct memory management and hardware-level operations. This versatility positions C++ between low-level languages like Assembly and modern high-level languages like Python, making it highly efficient and performant, particularly for computationally intensive tasks.

1.2.2 Learning Resources for C++

Given the breadth of the language, this course will focus on essential concepts and practices relevant to high-performance computing. For more comprehensive learning, the following resources

are highly recommended:

Online resources:

- https://cppreference.com/
  A precise and extensive reference for C++ language features and libraries, complete with examples.
- https://isocpp.github.io/CppCoreGuidelines/
  A guide to writing efficient, maintainable, and safe C++ code.

1.2.3 Repetition: Programming Languages

Before diving into C++, this section revisits some foundational concepts of programming to ensure a clear understanding of key terms and ideas.

Definitions:

- A *computer* is a device that executes programs.
- A *program* is a precise sequence of instructions, enabling a computer to perform a specific task.

How Computers Understand Instructions

At the core of programming is the interaction between a computer and the instructions it executes. To fully appreciate the role of programming languages like C++, it's crucial to understand how computers interpret and process instructions at the most fundamental level.

A computer fundamentally "understands" binary, as its electronic components operate based on two states: voltage present (1) or not present (0). This binary logic is foundational to digital computing, where thresholds are applied to distinguish between these states, ensuring resilience against fluctuations and noise. Boolean logic is then used to process these signals efficiently.

The exact binary instructions a computer understands depend on its hardware architecture. For instance, ARM-based processors interpret instructions differently than x86_64-based processors. Data is represented in binary, too, but its interpretation depends on the specified *data type*. Consider the following binary sequence:

```
01100101 01110110 01101001 01101100
```

- interpreted as a 32-bit float (IEEE-754) results in `7.272793e22`.
- interpreted as a 32-bit integer results in `1702259052`.

- interpreted as ASCII/UTF-8 text results in `"evil"`.

Understanding how computers work with binary at this level forms the foundation for learning how programming languages bridge the gap between human-readable instructions and machine-readable operations.

While computers operate on binary instructions, it would be impractical and time-consuming for humans to write programs directly in binary. High-level programming languages were developed to simplify this process, allowing humans to write instructions that are closer to natural language while maintaining the precision required for computers.

High-level programming languages provide an abstraction layer that shields programmers from the complexity of binary or low-level machine instructions. Unlike human languages, which allow for interpretative flexibility, programming languages are strictly defined with precise syntax and semantics. Each statement corresponds to a specific operation defined by the language's rules.

Since computers only "speak" binary, instructions written in high-level languages need to be translated into machine-readable binary instructions. This translation is typically performed by a *compiler* or an *interpreter*:

- A *compiler* translates the entire source code into a binary executable file at once. This executable can then be run repeatedly without needing the source code, as long as the target computer uses the same architecture.
- An *interpreter*, on the other hand, translates and executes source code line-by-line at runtime, without producing a standalone executable.

This distinction between compilers and interpreters highlights a key difference in how programming languages are implemented, which is particularly relevant when considering C++.

C++ as a Compiled Language

C++ is a high-level programming language that is almost[1] always implemented as a compiled language. This means that programs written in C++ are first translated into binary executable files by a compiler before they can be executed.

This compilation process gives C++ its reputation for efficiency and speed, as the resulting binary is optimized for the specific architecture on which it will run. Numerous vendors provide C++ compilers, including the Free Software Foundation, LLVM, Microsoft, Intel, Embarcadero, Oracle, and IBM. For this course, the recommended compilers are:

- `g++` (GNU Compiler Collection) by the Free Software Foundation.
- `clang++` by LLVM.

[1] Some Just-In-Time (JIT) compilers, such as the one included in the ROOT Framework developed at CERN, enable C++ to behave somewhat like an interpreted language.

By understanding the role of compilers and how they relate to C++, you'll be better equipped to write efficient programs and manage their execution across different systems.

1.2.4 Compiling C++ Source Code - Single Source File

As already mentioned, the compilation of source code is the translation from a programming language into machine instructions. For a single file, this process is more intuitive, so we begin with a simple example. Later, we will explore more complex projects in greater detail.

The Minimal C++ Program

Before diving into a full-fledged example, let's create the simplest possible C++ program, containing only an empty `main()` function. This serves as a starting point to demonstrate how to compile and execute C++ code.

Create a plain text file with the `.cpp` extension, named `Minimal.cpp`, and add the following content:

```
1 int main()
2 {
3     return 0;
4 }
```

Explanation:

- The `main()` function is the entry point of every C++ program. When the program is executed, this function is called first.
- The `return 0;` statement signals to the operating system that the program has executed successfully. Returning a non-zero value typically indicates an error.

To compile and execute this minimal program:

1. Compile the source code into an executable file using the `clang++` compiler:
   `clang++ Minimal.cpp -o Minimal.out`
2. Run the resulting executable:
   `./Minimal.out`
3. Since this program has no output, nothing will appear in the console.

This minimal example demonstrates the steps to compile and run a C++ program.

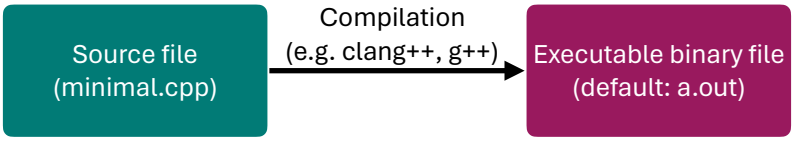

Figure 1.1: Translation of C++ source code into executable machine instructions.

The "Hello World!" Program

Now, let's expand on this by creating a more illustrative program. The "Hello World!" example introduces basic input and output functionality in C++. Start by creating another file, named `HelloWorld.cpp`, and add the following content:

```cpp
#include <iostream>

int main()
{
    std::cout << "Hello world!" << std::endl;
    return 0;
}
```

Explanation:

- The `#include <iostream>` directive imports the `iostream` library, enabling input and output operations.
- `std::cout` is used to output text to the console. The `<<` operator streams the string `"Hello world!"` to the standard output.
- `std::endl` adds a newline character and flushes the output buffer, ensuring the output appears immediately.
- The program ends with `return 0;`, signaling successful execution.

To compile and execute this program:

1. Compile the source code:
   `clang++ HelloWorld.cpp -o HelloWorld.out`
2. Run the executable:
   `./HelloWorld.out`

These two examples - `Minimal.cpp` and `HelloWorld.cpp` - provide a foundational understanding of compiling and running C++ programs.

Data Type	(Typical) Size in Bits
`bool`	1
`char`	8
`int`	32
`long`	64
`float`	32
`double`	64

Table 1.2: Example of a few fundamental data types and their typical sizes on modern PCs.

1.2.5 Primitive/Fundamental Data Types in C++

There are several *primitive* or *fundamental data types* in C++, such as those listed in Table 1.2, that help programmers to build more complex structures and data types.

Here, these data types are given with their typical sizes in bytes. In the C++ standard, sizes of fundamental data types are given as minimum required precision, not as a fixed size value. For example, the data type `int` requires at least 16 Bit / 2 Bytes, but usually comes with 32 Bit / 4 Bytes on modern computer architectures. More on that here: https://en.cppreference.com/w/cpp/language/types.

This decision is rooted historically and is influenced by the need for portability and compatibility (e.g. backward compatibility) across various platforms, including those with different word sizes and memory constraints, e.g. in embedded systems.

These sizes are sometimes neglected as long as the used data types are sufficiently large to store the data. In high-performance computing, however, these can have crucial factors with respect to the runtime. For example, the charge of a particle in heavy-ion physics experiments can have one of three different states: positively (+1), neutrally (0) or negatively (-1) charged. Therefore, a datatype that can store at least three different states is sufficient.

A bool with only two states (true, false) is not sufficient and the next largest integer type that can represent it is a `short` with 16 Bit on modern computers. Intuitively, this would make sense as it can easily store negative and positive numbers. However, a `char` has a size of 8 Bits and can also easily represent the three states. Although it is a character type, it can be used to store the charge of a particle. For each particle, there are 2 times less Bits required to store its electrical charge when using `char` in comparison to `short`.

But does it make a difference? Well, it depends. If the computer reads a line of aligned memory (RAM) and stores it into the cache, twice as much values can be stored when we would store the charges of particles as `char`. Access to the memory is quite costly, that sometimes even re-calculation can be much faster than reading it from memory. Just to provide some numbers: a memory access can easily take up to 100 ns, whereas L1 cache access can be as fast as 0.5 ns.

Nevertheless, to achieve an efficient runtime, optimal data types help definitely, but how they are stored and aligned in memory is at least as crucial as finding the best matching one. Later in this course, different types of storing data are discussed in more detail.

1.2.6 Types and Casting

C++ is a statically and strongly typed language, which means the types of variables must be known at compile-time. This attribute enforces type safety by generating errors if variable types do not match during compilation, limiting implicit conversions. It is imperative in C++ that every variable and function return type is explicitly declared. In C++ 11, the *auto* keyword was introduced, enabling the compiler to infer variable types at compile time, enhancing code readability and maintainability, especially in complex scenarios like iterating over containers or using lambdas.

Regarding type casting, C++ supports conversions between different types, categorized into implicit and explicit conversions. Implicit conversions are automatically performed by the compiler, whereas explicit conversions require specific syntax to instruct the compiler. Although C++ maintains backward compatibility with C-style casting, it is advisable to utilize C++'s four cast operators for clearer, type-safe conversions. This recommendation stems from the fact that C-style casting can allow conversions between incompatible types, posing risks for code maintenance and clarity, especially in collaborative environments.

The C++ cast operators are:

- `static_cast<T>`: Suitable for non-polymorphic type conversions, including upcasting and downcasting within class hierarchies, numeric type conversions, and pointer type conversions where implicit or reversible. This operator ensures type safety through compile-time checks.
- `dynamic_cast<T>`: Utilized for safe downcasting in class hierarchies, converting a base class pointer or reference to a derived class pointer or reference. It employs Run-Time Type Information (RTTI) to verify the cast's validity, returning a null pointer or throwing a `bad_cast` exception upon failure.
- `reinterpret_cast<T>`: Enables conversions between pointer types or between pointer and integer types, performing a bit-wise copy of the value. Its use is generally discouraged due to potential alignment and type punning issues, and should only be considered when absolutely necessary.
- `const_cast<T>`: The only cast capable of modifying the constness of an object, useful for invoking non-const methods on const objects or modifying objects initially declared as const. This operator can add or remove the `const`, `volatile`, or `__unaligned` qualifier from a pointer or reference.

Transitioning from the basics of types and casting, let's delve into practical examples to illustrate these concepts further.

Examples:

```
1 // different static type-casting variants
2 short a = 10;
3 float b;
```

```
b = a;                    // implicit (variant 0)
b = (float) a;            // explicit (variant 1)
b = float(a);             // explicit (variant 2)
b = static_cast<float>(a); // explicit (variant 3)

// dynamic type-casting for pointers/references to objects
Animal* animal = new Dog();
Dog* dog = dynamic_cast<Dog*>(animal);

// reinterpreting memory, here array interpreted as __m128 vector:
float data[] = {1.f, 2.f, 3.f, 4.f};
__m128 &vec = reinterpret_cast<__m128&>(data);

// modifying constness of an object to re-write const-objects:
int c = 10;
const int& d = c;         // create "write-protected" reference
const_cast<int&>(d) = 5;  // re-write "write-protected" reference
```

1.2.7 Variables, Literals, and Expressions

In C++, *variables* are identifiers providing a name to objects in memory, for instance, `int a;` defines a variable named `a` of type `int`. *Literals* represent constant values directly included in the code, like `5` in `a = 5;`. An *expression* involves operators and operands performing computations, such as `a++`, which increments `a`.

Each object in C++ is characterized not just by its data type and size, but also by two main properties:

- An address, indicating the memory location of the object.
- Content, the data value stored at the object's memory location.

```
int a = 42; // Creates an integer variable named 'a' with a value of 42
a = 1337;   // Modifies the content of 'a' to 1337
```

1.2.8 Initialization in C++

C++ supports various initialization methods. Traditional forms include:

- Direct Initialization: `int a(5);`
- Copy Initialization: `int a = 5;`

C++11 introduced *Brace Initialization*, offering advantages such as a uniform syntax across all types, prevention of narrowing conversions, and uniform initialization of aggregates (arrays, structs):

```
int a{4};      // No error
int b{7.9};    // Compiler error due to narrowing
```

Initialization of data structures:

```
std::vector<int> v{1, 2, 3};
int array[]{1, 2, 3};
YourClass obj{};
```

This initialization method enhances code safety and is thus recommended in modern C++ practices.

1.2.9 Pointers in C++

Your introduction effectively sets the stage for understanding memory addresses. Clarifying that every memory cell can be addressed and that variables are allocated to these addresses could be beneficial. It might be helpful to mention that the size and alignment of variables affect how they are placed in memory, which is managed by the compiler and the runtime environment.

Any variables in C++ can be accessed with its identifier (name). This is the easiest way when we don't need to care about the physical location of our data in the memory. However, there is a second way to access a variable, directly using its location in the memory. The computer's memory can be represented as a sequence of memory cells of one byte each. These cells are numbered in a consecutive way. Each cell has a unique number in the whole available memory. Every next cell has the number of the previous one plus one. This way we can claim that the cell number 55 definitely follows the cell number 54. As soon as we declare a variable, the amount it needs in memory is allocated in a specific location (memory address). The operating system performs this task automatically during runtime. This memory address locates a variable in the memory. This memory address can be obtained by adding an ampersand sign (`&`), the so-called *address operator*, in front of the identifier. So `&a` gets the address of the variable a. A variable which stores the memory address of another variable is called a pointer.

Pointers are a foundational aspect of C++, allowing variables to reference memory locations of objects. Key points regarding pointers include:

- Pointers have a consistent size, typically making them more efficient to move or copy compared to large data structures.
- Modifying an object via a pointer affects the original object globally.

– Pointers facilitate data structure manipulations, such as swapping nodes in trees without moving all child nodes.

Pointers are said to ”point to” the variable whose address they store. All pointers have a special pointer type. While declaring a pointer, one has to specify this type. The pointer type is obtained by adding an asterisk after the type it points to.

```
int a = 5;    // 5 stored at some memory location, let's say at 0x7ffcdf8a990c
int* b = &a;  // 'b' points to the address of 'a' (0x7ffcdf8a990c)
              // 'b' itself is located somewhere else, e.g. at 0x7ffcdf8a9900
```

The content of a pointer is a memory address and the address where the pointer is pointing to has some content, which can be accessed by dereferencing the pointer.

Dereferencing Pointers: The value (content) at the pointer's referenced memory location can be accessed by dereferencing the pointer. One can use the asterik sign (*) in front of the pointer variable to dereference the pointer. In this case, the * is called *dereference operator*.

```
std::cout << "address of a: " << &a << ", content of b: " << b << std::endl;
std::cout << "content of a: " << a << ", dereferenced b: " << *b << std::endl;
std::cout << "address of b: " << &b << std::endl;
```

Additionally, operations can be applied to pointers, like arithmetic operations to traverse arrays.

```
int arr[2];
int* p = arr; // same as "int* p = &arr[0]" -> p points to the first element
++p;          // p points now to the second element
```

In this example, the behavior of `arr` is similar to pointer: one can either set `p` to `arr` or to the address of element 0 of the array, but not to the address of `arr`. It might seem like the content of `arr` is an address, similar to pointers, but actually `arr` is a array-type. But arrays in expressions decay to pointers to their first element, reflecting a design choice from C for simplicity and efficiency.

This conversion facilitates passing arrays to functions as pointers, enabling efficient data manipulation. However, a critical implication of array decay is the loss of size information. When an array decays to a pointer, the function receiving the pointer does not inherently know the size of the original array. This necessitates careful management, often requiring the size of the array to be passed as an additional parameter to functions that operate on the array. Understanding and accounting for array decay is essential to avoid out-of-bounds access, ensuring safe and effective use of arrays in C++. In modern C++, other container classes/structs are usually used, such as `std::vector<T>` or `std::array<T>`.

Fundamentally, a pointer is designed to hold the address of a single memory location. However, when it references an object occupying more than one byte in memory – such as a multi-byte data type or an array – the pointer is actually pointing to the initial byte of a contiguous block of memory where the object resides. For instance, a pointer referencing a 4-byte integer, or an array thereof, essentially points to the first byte of this integer. When such a pointer is incremented, it advances to the next element in the sequence. The magnitude of this advancement, or ”jump”, is determined by the size of the object type to which the pointer refers. Consequently, for an array of 4-byte integers, incrementing the pointer results in it moving 4 bytes forward, aligning with the start of the subsequent element in the array. Thus, pointer-arithmetic has type-awareness.

Please note: The `this` keyword in C++, representing the object itself, is also a pointer.

1.2.10 References in C++

References in C++ create an alias for an existing variable, serving as a powerful tool in high-performance computing by circumventing the need for costly copies. They find their primary utility in two scenarios: as return types and as function parameters, significantly enhancing performance by operating directly on the original objects.

Consider the distinction between working with a copy versus a reference:

Working with a copy:

```
Matrix matrix = obj.GetLargeMatrix(); // Incurs significant overhead by copying
matrix.DoSomething();
```

Working with a reference:

```
Matrix& matrix = obj.GetLargeMatrix(); // Efficiently accesses the original
matrix.DoSomething();
```

To utilize a reference, one has to change the function’s return type to return a specific reference type, e.g. `Matrix& GetLargeMatrix()`.

This technique is similarly applied to function parameters to ensure that operations are performed on the original data without incurring the overhead of copying. Function parameters declared with an &-sign are treated as references, enabling direct manipulation of arguments as if they were the originals. This approach is not only efficient but also maintains code clarity and intuitive logic flow. However, it is crucial to ensure that returned references do not point to local objects within a function scope to avoid dangling references and the errors resulting from this.

```
Matrix MatrixMultiplication(const Matrix& a, const Matrix& b)
```

Here, two references of type `Matrix` are expected as function parameters. In a non-static function that is applied directly to an object, one matrix (e.g. Matrix 'a') could be the object the function is applied to and then only one matrix (e.g. Matrix 'b') would be given as a parameter.

However, this is somewhat a design choice. In some cases, in high-performance computing, you will also see that the result reference is given as a function parameter. For example, when memory is already pre-allocated for re-using it multiple times, such a concept makes sense and avoids multiple re-allocations of large spaces in memory. In the end, this is usually a trade-off between intuitive code and performance. For maintenance reasons, you should not always opt for performance, but in limited environments, it might help.

The return type here is not a reference, as in this example, it is assumed that the result is a new matrix and not one of the two operands is overwritten. When creating objects inside a function body that is then not stored in another object with a longer lifetime, one can not return a reference to that object. When the function body (scope) ends, the lifetime of the new object ends and the reference would be referring to an object that does not exist anymore, leading to an error.

Additionally, in the example above, the `const` keyword is used to avoid accidental modifications to the matrices. The keyword will be discussed in more detail later on.

A reference is implemented similarly to a pointer (e.g. it has a small size of 8 bytes in 64-Bit systems), while a copy is always depending on the target object's size.

When working with large data structures, copying a large object is timely expensive and should be avoided whenever possible.

1. Memory has to be allocated for the new object
2. The whole object has to be copied into the new location

Avoiding unnecessary copies not only saves time but also reduces memory usage, which is critical in high-performance computing and memory-constrained environments like embedded systems.

1.2.11 Const Correctness

Ensuring `const` correctness in function parameters and return types is a best practice in C++ programming. This involves using `const` with references to prevent modification of the original object, enhancing code safety and expressiveness.

Example:

```
1 void DisplayMatrix(const Matrix& matrix) {
2     // Display the matrix without modifying it
3 }
```

Applying `const` to reference parameters enforces immutability, allowing read-only access to the

object. This guarantees that the function will not alter the object's state, providing clarity to other developers and preventing unintended side effects.

Furthermore, `const` correctness aids in the optimization of compiler-generated code, potentially unlocking performance enhancements through the assurance that certain objects remain unchanged throughout their use. This can also have a non-negligible effect on the runtime.

Const References as Return Types

While returning a reference can significantly optimize performance by avoiding unnecessary copies, combining it with `const` ensures the returned object is protected against unintended modifications. This technique is particularly useful when returning objects that are members of a class or have a static lifetime, ensuring the safety and integrity of the returned reference.

Example:

```
const Matrix& GetReadOnlyMatrix() {
    static Matrix matrix;
    // Perform operations
return matrix; // Returns a read-only reference
}
```

Incorporating `const` correctness into your C++ programming practices promotes not only efficiency and resource conservation but also contributes to the development of robust, error-resistant code.

1.2.12 Control Flow

C++ allows the programmer to have different paths of execution through the program. This is achieved via a collection of control flow statements. The simplest example is perhaps the if-statement that lets us execute a statement only if a conditional expression is true. For ease of understanding we will divide control flow statements into six categories: conditional statements, jumps, function calls, loops, halts and exceptions.

Conditional statements cause a block of code to be executed if and only if a certain condition, such as an equality (==) or inequality (>,<, <= etc.), is satisfied. The if-else and switch statements fall into this category. Jump statements tell the CPU to execute statements at some other location in the program. The break, continue and goto statements are jump statements. Loops tell the program to repeatedly execute some sequence until some condition is met. The while, do-while, for and ranged-for statements are used to generate loops. Exceptions are a special kind of statements designed for error handling such as unexpected user input. C++ uses try, throw and catch for exception handling. You will not encounter many uses of exceptions in this course, but the exception handling is important generally. Function calls are jumps to some other location in the code and back. Function calls and return are used together to achieve this effect. Finally, In some situations the only recourse is to terminate the program. Halt statements like std::exit() and

std::abort() statements can be used for this.

C++ allows the programmer to nest control flow statements i.e. it is allowed to use, for example, loops inside loops which may themselves be inside of an if statement. This nesting of control flow statements allows the program to take highly convoluted and input dependent paths. A good command over control flow statements is absolutely necessary to follow the rest of this course. Please spend time to master these statements.

The following example provides at least some applications of conditional statements and loops.

```
int a = 5;
int b = 10;
int c = (a > b) ? b : a--;

for (int i = 0; i < c; i++) {
    do {
        std::cout << "a: " << a << std::endl;
        a--;
    }
    while (a > -1);

    std::cout << "i: " << i << std::endl;
}
```

1.2.13 Stack vs. Heap

There are primarily two types of memory allocations in C++: stack and heap allocations. For stack allocation, the allocation happens on contiguous blocks of memory. We call it a stack memory allocation because the allocation happens in the function call stack. The size of memory to be allocated is known to the compiler and whenever a function is called, its variables get memory allocated on the stack. And whenever the function call is over, the memory for the variables is deallocated. A programmer does not have to worry about memory allocation and deallocation of stack variables. In the case of heap, the memory is allocated during the execution of instructions written by programmers. Every time when we make an object using the operators for dynamic allocation, it is always created in heap-space and the referencing information to these objects is always stored in stack-memory. Heap memory allocation is not as safe as Stack memory allocation because the data stored in this space is accessible or visible to all threads. If a programmer does not handle this memory well, a memory leak can happen in the program.

```
int a = 5;              // int allocated on stack
int* b = new int(3);    // int allocated on heap

// print out "5 3":
```

```
std::cout << a << " " << *b << std::endl;

delete b;               // free heap-allocated memory manually
```

1.2.14 Classes and Structs

Classes and structs are the constructs whereby the user can define their own types. The two constructs are identical in C++ except that in structs the default accessibility is public, whereas in classes the default is private. The term accessibility here refers to the allowed access to the members of the class/struct. Access controls enable you to separate the public interface of a class, which can be interacted with "directly" after an object from the class is created, from the private implementation details, which are set during object creation and changed using member functions of the class, and the protected members that are only for use by derived classes. The access specifier applies to all members declared after it until the next access specifier is encountered. Classes and structs can both contain data members and member functions, which enable you to describe the type's state and behavior. Structs, however, are often used to create small data structures, whereas classes are often used for more complex objects.

```
struct Node {
    int id;
    int value;
};

struct Edge {
    Node* u;
    Node* v;
};


class Graph {
private:
    int nNodes_;
    int nEdges_;

    Node* nodes_;
    Edge* edges_;
protected:
    // member functions and variables
public:
    Graph(Node* nodes, Edge* edges, int nNodes, int nEdges) :
        nodes_(nodes),
        edges_(edges),
        nNodes_(nNodes),
        nEdges_(nEdges)
    {
```

```
        // other constructor code
    }

    // member functions and variables
};
```

When working with classes in multiple header and source files, it is important to set guards to ensure that every class is included once. This can be done by the following guards:

```
#ifndef CLASSNAME_H
#define CLASSNAME_H

class CLASSNAME
{
    // class code here
};

#endif // CLASSNAME_H
```

1.2.15 Templates

The template system is designed to simplify the process of creating functions (or classes) that are able to work with different data types. Instead of manually writing a bunch of almost identical functions or classes (one for each set of different types), we can instead create a single template. Just like a normal definition, a template describes what a function or class looks like. However unlike a normal definition (where all types must be specified), a template uses one or more placeholder types. A placeholder type represents some type that is not known at the time the template is written, but that will be provided later. Once a template is defined, the compiler can use the template to generate as many overloaded functions (or classes) as needed, each using different actual types! Thus we only have to create and maintain a single template, and the compiler does all the hard work by replacing the types respectively. In fact, the C++ standard library itself is absolutely full of template code.

To create a template function, replace instances of specific types (such as int or float) with type template parameters which we name, for instance, T. Now we have to declare the template parameter to let the compiler know that the function is a template. We do this by adding the following line before the function declaration: template <typename T>.

```
// function template
template <typename T>
T GetMax (T a, T b) {
    if (a > b) return a;
    return b;
```

```
}

// class template
template <class T>
class Pair {
private:
    T a, b;

public:
    Pair (T first, T second) : a(first), b(second)
    {
        // other constructor code
    }

    T GetMax()
    {
      if (a > b) return a;
      return b;
    }
}
```

1.2.16 Multi-File Projects and (Meta-)Build-Tools

Many projects consist of more than a single source code file. There are several tools that can be used to handle projects. A well-known tool is CMake, that allows to create a configuration file with all relevant information, such as files to compile, compiler settings, dependency checks and much more. In the following example, we assume a project in the `project` directory. In this directory, there is the CMake configuration `CMakeList.txt` that contains all relevant information required for compiling the application. Depending on the project structure, there are code files or other directories with the code included that are then loaded by the `CMakeList.txt`.

To compile our project, a `build` directory is created using the "make directory"-command `mkdir`. A second step is to navigate into this directory using "change directory"-command `cd`. Then we use the CMake tool to setup our code for compilation, using the `CMakeList.txt` in the directory outside of the build directory - CMake will automatically search for it. After this, we can run `make` to build our project and run our application.

```
username@hostname:~/project$ mkdir build
username@hostname:~/project$ cd build
username@hostname:~/project/build$ cmake ../.
username@hostname:~/project/build$ make
username@hostname:~/project/build$ ./PROJECTNAME
```

A simple `CMakeLists.txt` file could look like this:

```
cmake_minimum_required(VERSION 3.22)
project(PROJECTNAME)

set(CMAKE_CXX_STANDARD 14)

include_directories(.)

add_executable(PROJECTNAME
    Project.cpp
    ClassA.h
    ClassA.cpp
    ClassB.h
    ClassC.cpp)
```

Chapter 2

Neural Networks

2.1 Introduction

Neural networks, also known as Artificial Neural Networks (ANNs), are a subset of machine learning and are a method in artificial intelligence that teaches computers to process data in a way that is inspired by the human brain. It is a type of machine learning process that uses interconnected nodes (neurons) in a layered structure, similar to neurons and synapses in a brain. It creates an adaptive system that computers use to learn from their mistakes and improve continuously. Thus, ANNs attempt to solve complicated problems, like summarizing documents or recognizing faces with high accuracy, and build the heart of deep learning algorithms.

Neural networks, along with other machine learning algorithms, have become popular across various scientific disciplines. A fundamental example of neural networks is the Multi-Layer Perceptron (MLP), which is often deemed a foundational neural network, since many advanced neural network models derive from it. This MLP has its roots in the simple perceptron, widely recognized as the cornerstone of neural network theory.

MLPs comprise of node layers, containing an input layer, one or more hidden layers, and an output layer. Each layer stores information or basically, a set of values, and each of the values is called a neuron. Each neuron is connected to at least one other neuron and has an associated weight and a threshold to be active or not. The output of each node is then activated before passing on to the next layer. A common type of activation used is to check if the output of any individual node is above the specified threshold value. In this case, that node is activated, sending data to the next layer of the network. Otherwise, no data is passed along to the next layer of the network. Modern neural networks, however, often apply mathematical functions to neurons or layers that modify the output values respectively to get a desired effect.

Neural networks rely on training data to learn and improve their accuracy over time. However, once these learning algorithms are fine-tuned for e.g. accuracy, they are powerful tools of computer science, allowing us to classify and cluster data at a high velocity. Tasks in speech recognition or image recognition can take minutes versus hours when compared to the manual identification by human experts. One of the most well-known neural networks is Google's search algorithm.

2.2 Multilayer Perceptrons

Among the various types of ANNs, in this chapter, the focus is on Multi-Layer Perceptrons (MLPs) with backpropagation based learning algorithms. These belong to the supervised learning, a subset of machine learning where the desired output for training is known exactly and that allows to provide accurate feedback for the network by the true labels (target values). It means that the network builds a model based on examples in data with known true outcomes. An MLP has to extract this relation solely from the presented examples, which together are assumed to implicitly contain the necessary information for this relation. The known true output is then used to provide feedback and adjust the weights accordingly.

As mentioned at the beginning an MLP comprises of three types of layers (input, hidden, and output) with nonlinear computational elements (neurons). The information flows from the input layer to the output layer through none, one or more hidden layer(s). In the common MLPs, the layers are often fully-connected layers. This means that all neurons from one layer are connected to all neurons in the adjacent layers as shown in Figure 2.1. These connections are represented as weights in the computational process. The weights play an important role in the propagation of the signal in the network. They contain the knowledge of the neural network, about the problem-solution relation. The number of neurons in the input layer depends on the number of independent variables in the model, whereas the number of neurons in the output layer is equal to the number of dependent variables. The number of output neurons can be scalar, vector or higher dimension.

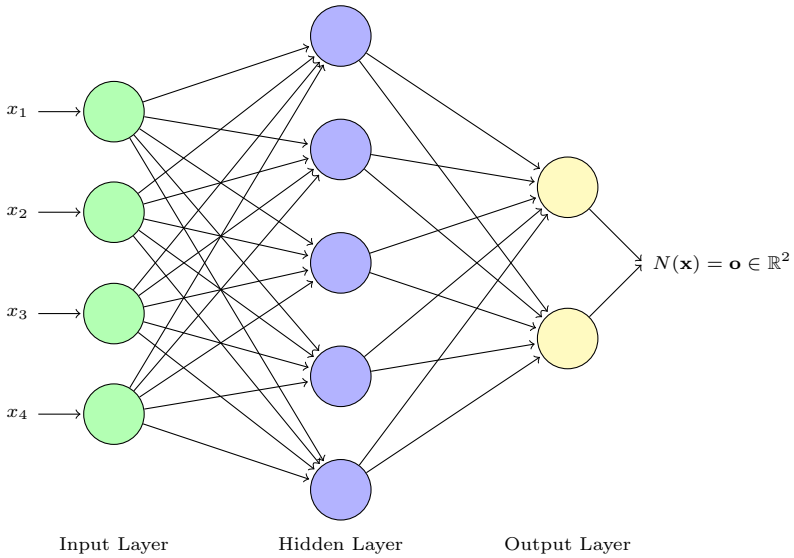

Figure 2.1: MLP with 4 input neurons, 5 hidden neurons and 2 output neurons.

2.3 Learning Algorithm

An MLP is trained/learned to minimize errors between the desired target values and the values computed from the model. If the network gives the wrong answer, or if the errors are greater than a given threshold, the weights are updated to minimize them. Thus, errors are reduced and, as a result, future responses of the network are likely to be correct. In the model we will

be learning based on this chapter, the network will be updated, more specifically the weights would be updated, for both the correct and incorrect results so as to reach a global minimum with respect to its prediction capabilities. In the learning procedure, datasets of input and desired target pattern pairs are presented sequentially to the network. The learning algorithm of an MLP involves a forward-propagation step followed by a backward-propagation (BackProp) step.

2.4 Forward Propagation

The forward-propagation phase begins with the presentation of the input variable(s) **X** (which can be a vector, matrix or any order tensor) to the input layer. As in biological neural systems, in which dendrites receive signals from neurons and send them to the cell body, an MLP receives information through input and output neurons and summarizes the information. MLP training is based on an iterative gradient algorithm to minimize error between the desired target and the computed model output. The neuron *j* of the hidden layer shown in the figure below is calculated from the previous layer, where the summation of each output of the previous layer (the input layer in the figure below) multiplied by weight w_{ij} (which is the weight associated with the *i*th component of the previous layer and *j*th component of the current layer). However, note that this is not the final output of the neuron h_i.

$$a_j = \sum_{i=N_{input}} x_i^{input} w_{ij}$$

where N_{input} is the number of features of the input layer (you can also call them features, units, nodes etc).

In general we also add a *bias* to our calculation.

$$a_j = \sum_{i=N_{input}} x_i^{input} w_{ij} + b_j$$

This is followed by the application of an activation function and the final output becomes the neuron j of the hidden layer and will be used in the construction of the next layer. The activation function is almost always a non-linear function. It plays an integral role in neural networks by introducing nonlinearity. This nonlinearity allows neural networks to develop complex representations and functions based on the inputs that would not be possible with a simple linear regression model. Some of the most common activation functions are: sigmoid, tanh, and ReLU. All hidden layers usually use the same activation function. However, the output layer will typically use a different activation function from the hidden layers. The choice depends on the goal or type of prediction made by the model. Assume that f_A is some activation function. Then,

$$h_j = f_A(a_j)$$

For all the N_{hidden} units in the hidden layer we can represent the above summation in a compact form as such:

$$\mathbf{a}_{hidden} = \mathbf{W}_{hidden} \cdot \mathbf{x}_{input} + \mathbf{b}_{hidden}$$

and the activation function which follows as :

$$\mathbf{h}_{hidden} = F_A(\mathbf{a}_{hidden})$$

Here, the activation is applied piece-wise if $\mathbf{a}$ has components.

In general we can say that the units/features(as mentioned before they are also called neurons, nodes etc) of *n*th layer, $\mathbf{h}_n$ can be written as:

$$\mathbf{a}_n = \mathbf{W}_n \cdot \mathbf{h}_{n-1} + \mathbf{b}_n$$
$$\mathbf{h}_n = F_A(\mathbf{a}_n)$$

2.5 Loss Function

In general, for any optimization problems, we have an objective function that we minimize or maximize in order to obtain our desired output. For the MLP that we will be learning, we will be minimising this objective function and as such we will be calling it the *loss* function. You can also see terms like *cost*, *error* in other sources of text. So these terms will be used interchangeably in the following.

One of the simplest loss function is to get the predictions from the network, subtract it from the actual number (the true value) we wanted to get and square it:

$$\mathbf{L}(y, \hat{y}) = \frac{1}{N}\sum_{i=1}^{N}(y - \hat{y})^2$$

where *N* is the total number of training data. In practice, and in the exercises, we will be using Loss functions over a sample of training data, referred to as *batch size*. More details on this will be given in the section on *Gradient Descent*

2.6 Backpropagation

Backpropagation is the algorithm for fast calculation of the gradient with respect to the weights and bias, so that they can be later optimized based on a gradient-based optimization procedure. The main goal of the backpropagation algorithm is to calculate the gradients or partial derivatives of the loss function with respect to the weights and bias in each layer.

For example: $\frac{\partial C}{\partial w_{ij}}$, $\frac{\partial C}{\partial b_j}$ are the gradients with respect to the weight associated with neuron *j* mentioned in the *Section 2.4* and its bias.

Since the gradients are calculated based of the loss function, it's calculation starts from the final layer, i.e, the output layer. After that, these output layer gradients are used to calculate the gradients of the layer before the output layer. This is repeated until the layer after the input layer. Since the input layer is the data, it doesn't have any weights or bias for its construction. In order to calculate these gradients from the final output layer, where the cost function is calculated, to the lower layers, we use the chain rule to derive lower gradients from higher ones.

$$\frac{\partial C}{\partial x_i} = [\frac{\partial C}{\partial x_n}, \frac{\partial x_n}{\partial x_{n-1}}, ..\frac{\partial xi+1}{\partial x_i}]$$

So, to demonstrate the overall the backpropagation algorithm, consider an MLP with

$$(N+2)$$

layers, which are more specifically, 1 input layer (the 0-th layer), N hidden layers and 1 output layer (the $(N+1)$-th layer). Let $L(X)$ be the *loss* function.

1. Compute the gradient on the output layer:
   $g = \nabla_y L$
   Calculate the gradient for the output layer and all lower levels.

2. **for** $n = N+1$ to 1 **do**

   - Calculate gradient based on the layer features before activation. For this, just multiply the gradient of the layer with the derivative of the activation.
     $g = \nabla_{a_n} L = g \odot f'(a_n)$
     For eg:-, incase of the output layer in our example, this is just:
     $g = \nabla_{a_{output}} L = g \odot f'(a_{output})$
     , where the g on the RHS is gradient from the output layer($g = \nabla_y L$)

   - From this gradient we can calculate the gradient of the weights and bias used to create/construct this layer:
     $\nabla_{W_n} L = \nabla_{a_n} L \odot \nabla_{W_n} a_n = g \odot \nabla_{W_n} a_n$
     If we look back to **Section 1.2** on forward propagation, we can see that:

     $$\nabla_{W_n} a_n = g \odot \mathbf{h}_{n-1}$$

     $$\nabla_{b_n} a_n = g$$

     (Easily derivable from

     $$\mathbf{a}_n = \mathbf{W}_n \cdot \mathbf{h}_{n-1} + \mathbf{b}_n$$

)

   - Calculate gradients for the next layer:

     $$\nabla_{h_{n-1}} L = g \odot \nabla_{h_{n-1}} a_n$$

     $$\rightarrow \nabla_{h_{n-1}} L = W_n^T \cdot g$$

This summarizes the BackProp algorithm that we will be using for learning the MLP. For other type of neural networks, the main factor which changes is the formula for the gradient and change in the gradient formulation arising from the way the layer is constructed(for MLP it is simple matrix multiplication as seen in *Section 2.4*). At the core, we use chain rule and start from the final output layer and work our way backwards till all the gradients are calculated.

2.7 Gradient Descent

The neural networks that we will be learning are trained using the stochastic gradient descent algorithm. Stochastic gradient descent is an optimization algorithm that estimates the error gradient for the current state of the model using examples from the training dataset, then updates the weights of the model using the gradients calculated from backpropagation. So, we would have the form:

$$w_{ij} = w_{ij} - \eta \frac{\partial L}{\partial w_{ij}}$$

where w_{ij} again refers to the weight used in *Section 1.2*. This is a general formula and applies to all weights and bias and η is the learning rate.

The amount that the weights are updated during training is referred to as the step size or the "*learning rate*". Specifically, the learning rate is a configurable hyperparameter used in the training of neural networks that has a small positive value, often in the range between 0.0 and 1.0. The learning rate controls how quickly the model is adapted to the problem. Smaller learning rates require more training epochs given the smaller changes made to the weights each update, whereas larger learning rates result in rapid changes and require fewer training epochs. A learning rate that is too large can cause the model to converge too quickly to a suboptimal solution, whereas a learning rate that is too small can cause the process to get stuck.

In theory, the gradient calculation is done over the complete training data set. This is also known as *Batch Gradient Descent*. However, in this case, each epoch will only have one update. Although, this is guaranteed to converge to the Loss minima, we will have many epochs for this to happen. This can be very inefficient and slow because each iteration of the algorithm only improves our loss by some small step. There's also *Stochastic Gradient Descent* where we update after each example is passed through the neural network. Although, this method converges faster, it will more fluctuations in the cost function and the computations cannot be parallelized efficiently since we are using one example at a time.

In our exercise, and in most implementations, we use *mini-batch Gradient Descent* to solve this problem. It combines the capabilities of both *Batch Gradient Descent* and *Stochastic Gradient Descent*. The rule for updating the weights stays the same, but we're going to take on some small mini-batch of the dataset and use the mean gradient on that to update the weights. (**Note**: Some texts refer to *mini-batch Gradient Descent* as *Stochastic Gradient Descent* and use them interchangeably. We will do the same).

2.8 Important Key Words and (Hyper-)Parameters

Although machine learning isn't the primary focus of this lecture, it's beneficial to understand some key terms that frequently appear in this domain.

These terms include:

- Number of epochs, batch size, learning rate, and other hyperparameters
- Model descriptions
- Dataset splitting strategies for training, validation, and testing
- Underfitting and overfitting in model training
- Various parameter optimization techniques beyond standard gradient descent
- Regularization methods to improve generalization
- And more…

The following subsections provide definitions and explanations for each of these concepts.

2.8.1 Epochs

The term *epoch* refers to a single cycle through the entire training dataset. Since data is often limited, the model undergoes multiple epochs, adjusting weights iteratively to learn patterns within the data. Sometimes, a ”validation” or ”testing” phase is also conducted each epoch to evaluate performance on unseen data per epoch to see how the behavior on unseen data is, but a final test should be done with entirely new unseen data before model deployment.

Advanced optimization techniques like decaying learning rates benefit from shuffling data each epoch to prevent overfitting. Otherwise, the same data in the beginning of the dataset would always be learned with a higher learning rate then data in the middle or the end of the dataset.

2.8.2 Learning Rate

The *learning rate* determines the step size for each weight update. It multiplies the gradient value to control how much the model adjusts. Smaller learning rates allow for finer adjustments but may lead to longer training times and a risk of getting stuck in local minima. Larger learning rates may speed up training but risk overshooting optimal values, potentially never converging to a good solution.

2.8.3 Batch Size

The *batch size* is the number of samples processed before updating the model’s parameters. Different batch sizes lead to different gradient descent approaches:

- *Batch Gradient Descent:* Processes the entire dataset before updating weights, leading to high memory usage and slow progress.

- *Stochastic Gradient Descent:* Processes one sample at a time, resulting in faster updates but high variance in gradients.
- *Mini-Batch Gradient Descent:* Balances both by processing a small subset, allowing parallelization and smooth progress.

2.8.4 Defining Layers in Neural Networks

The rapid growth of machine learning research, particularly in neural networks, has led to inconsistencies in terminology due to the absence of universal conventions. A common example of this is the definition of a "layer." Some sources define a layer as a set of neurons without considering the connections between them, while others include weight matrices or even the outputs that feed into the next layer. This can lead to confusion, especially in cases where activation functions are not applied directly to input features. For example, if there's no activation function on the input features, should we consider them an input layer? Maybe not, but they are multiplied by a weight matrix and generate outputs for the next layer. Assuming an identity activation function on input features would mimic the behavior of typical MLP layers, so we might reasonably classify it as a layer.

In PyTorch, for instance, a linear layer is specified with `nn.Linear(in_features, out_features, ...)`. By this definition, we have a multi-layer perceptron with input features, a weight matrix, and output features. Combined with an activation function, we obtain a full neural network without hidden layers. Here, we might consider only one MLP layer, but if we view the input features as implemented neurons, we might interpret this setup as having two layers.

Ultimately, the interpretation depends on the context. In PyTorch, we might simply say we have one linear layer, not counting the input layer as a true "layer." Alternatively, in a more descriptive context, we might refer to these as input, hidden, and output layers.

For this lecture, we'll count each set of neurons as a distinct layer, whether it's designated as an input, hidden, or output layer. Regardless of approach, it's helpful to clarify definitions upfront to ensure consistent understanding in any specific context.

2.8.5 Dataset Splits

Datasets are commonly split into training, validation, and testing subsets:

- *Training Set:* Used to train the model, with the aim of capturing general patterns rather than memorizing specific examples.
- *Validation Set:* Used for tuning hyperparameters and evaluating different models during training, helping to select the best-performing version.
- *Test Set:* Reserved for final model evaluation to assess performance on unseen data, reflecting generalization ability.

2.8.6 Underfitting and Overfitting

Underfitting occurs when a model is too simple to capture the underlying patterns in the data, leading to poor performance on both training and test data. This typically occurs when the network is initialized with random weights but has not yet trained, or after training if the model lacks sufficient parameters to learn the underlying data patterns.

Overfitting happens when the model learns too many details of the training data, failing to generalize to new data. This often happens when using too many training iterations without applying optimization methods to prevent overfitting, or when the model has so many parameters that it memorizes the training data.

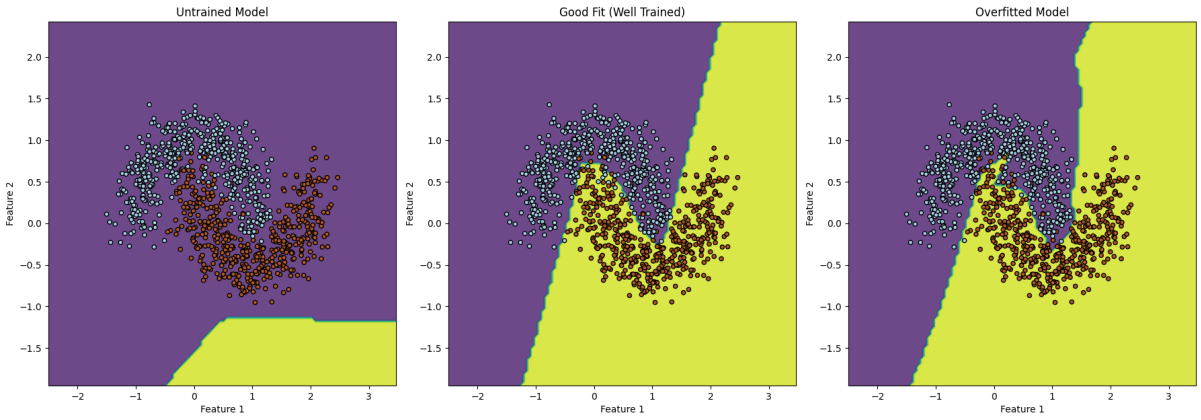

Figure 2.2: Classification example with an underfitted (untrained) model, a robust model and an overfitted model. The background color shows the decision space for one class or another. While the problem for the underfitted model is obvious, it is different for the overfitted one. There, the model tries to catch data points that reach into the cloud of the other class and thus, likely learning noise from that data. Adding more data will show for this example that these data points are ”outliers”. Trying to catch them, decreases the robustness for new unseen data.

2.9 Optimization Techniques

2.9.1 Gradient Descent Variants

The basic gradient descent algorithm adjusts model weights based solely on gradient information. However, specific issues, such as getting stuck in local minima or overshooting minima, have led to several gradient descent variants:

- *Look-Ahead Methods:* Help avoid local minima by considering future steps.
- *Decaying Learning Rates:* Adjust learning rate over time to improve convergence.
- *Momentum-Based Updates:* Use past gradients to help navigate sparse features and handle zero-value paths.
- *Random Dropout:* Regularly drops random neurons during training, improving generalization and preventing overfitting.

2.9.2 Regularization Methods

Regularization techniques improve model generalization by discouraging complexity in the learned model:

- *L1 Regularization (Lasso):* Adds a penalty proportional to the sum of absolute weights. This encourages sparsity, leading to some weights becoming zero, which can simplify the model.
- *L2 Regularization (Ridge):* Adds a penalty proportional to the sum of squared weights. This discourages large weight values, preventing overfitting while retaining all features.
- *Dropout:* Randomly deactivates neurons during training, helping to prevent co-adaptation of features and enhancing generalization.

These methods can be adjusted based on the dataset and desired model behavior.

Chapter 3

Parallelization Methods

3.1 Moore's Law

Moore's Law is a prediction made by Gordon E. Moore, co-founder of Intel Corporation, in 1965. The law states that the number of transistors, resistors, diodes and capacitors on a microchip will double every 18 to 24 months, which leads to a rapid increase in computing power while decreasing the cost per component. Moore's Law has had a significant impact on the development of High Performance Computing (HPC) by enabling the creation of increasingly powerful and complex electronic devices that can handle large-scale scientific simulations and data-intensive computations. In the context of HPC, Moore's Law has been a driving force behind the development of faster and more powerful microprocessors, which have enabled researchers to perform simulations and calculations that were previously impossible. For example, the increase in computing power has allowed scientists to simulate complex physical phenomena like climate change, astrophysics, and molecular dynamics, which require massive amounts of computational power. However, recently, there are signs that Moore's Law may be reaching its limits. The physical limitations of semiconductor manufacturing technology, such as the size of the transistors, are becoming more apparent. As transistors become smaller and more densely packed, they generate more heat, which can lead to problems with reliability and performance. Furthermore, the cost of developing and manufacturing increasingly complex microchips is increasing, making it harder for companies to keep up with the pace of Moore's Law. As a result, some experts predict that the rate of growth predicted by Moore's Law may slow down or even come to a halt in the near future. This could have implications for the development of HPC, as it may become increasingly challenging to develop the more powerful processors required for large-scale simulations and computations. Despite these challenges, there are still efforts to keep Moore's Law alive, such as the development of new materials and manufacturing techniques. Additionally, there are emerging technologies such as quantum computing and neuromorphic computing that may enable the continuation of exponential growth in computing power beyond the limits of Moore's Law.

3.2 The Need for Parallelization

As the number of cores in a processor increases, the frequency of each core typically decreases to maintain thermal design power (TDP) limits. This presents a significant challenge for com-

putational performance, as many applications are designed to run on a single thread with high frequency. One solution to this challenge is to develop applications that can take advantage of multiple cores through parallelization. Parallelization involves breaking down a task into smaller sub-tasks that can be processed simultaneously on multiple cores. This can significantly increase performance by allowing the processor to perform more work in the same amount of time. However, parallelization is not always easy to implement, and not all applications are easily parallelizable. In some cases, parallelization is not possible due to dependencies or may require significant changes to the code or the use of specialized libraries or tools. Another solution to this challenge is the use of hardware accelerators such as Graphics Processing Units (GPUs). These devices are designed to perform specific types of computations much faster than a traditional CPU, making them ideal for computationally intensive tasks such as deep learning or scientific simulations. GPUs can also be used in conjunction with CPUs to offload certain types of computations and improve overall performance. For example, GPUs can be used to perform the computationally intensive parts of a simulation, while the CPU handles the rest of the computation. In general, parallelization or use hardware accelerators like GPUs are some solutions to the computational challenges posed by more cores with the same frequency.

3.3 Flynn's Taxonomy

Flynn's taxonomy is a categorization of forms of parallel computer architectures by Michael J. Flynn. Parallel computers are classified by the concurrency, or simultaneous calculations, in processing sequences (or streams), data, and instructions. This results in four classes:

- **Single Instruction Single Data (SISD)**
  A sequential computer which exploits no parallelism in either the instruction or data streams. Single control unit (CU) fetches single instruction stream (IS) from memory. The CU then generates appropriate control signals to direct single processing element (PE) to operate on single data stream (DS) i.e. one operation at a time. The traditional uniprocessor machines like a PC (currently manufactured PCs have multiple cores) or old mainframes.

- **Single Instruction Multiple Data (SIMD)**
  A computer which exploits multiple data streams against a single instruction stream to perform operations which may be naturally parallelized. For example, an array processor or graphics processing unit (GPU). This can be combined with other task-level parallization techniques to get further speedups.

- **Multiple Instructions Multiple Data (MISD)**
  Multiple instructions operate on one data stream. Uncommon architecture which is generally used for fault tolerance. Heterogeneous systems operate on the same data stream and must agree on the result. Examples include the Space Shuttle flight control computer.

- **Multiple Instructions Multiple Data (MIMD)**
  Multiple autonomous processors simultaneously executing different instructions on different data. MIMD architectures include multi-core superscalar processors, and distributed systems, using either one shared memory space or a distributed memory space.

3.4 Types of Parallelization

Parallelization is a technique that involves breaking down a task into smaller sub-tasks that can be processed simultaneously on multiple cores or threads. There are several types of parallelization using threads and cores. However, to fully take advantage of these advanced programming models, we need to understand the basics of both paradigms. Threads and cores are both components of a computer's processing unit, but they serve different functions and have different capabilities.

Thread: A thread is a unit of execution within a process. Threads share the same memory space as their parent process and can execute instructions concurrently with other threads within the same process.

Core: A core is a physical processing unit that can execute instructions independently. It contains an arithmetic logic unit (ALU) for performing arithmetic and logical operations, as well as a control unit for managing the flow of instructions. A computer processor can have multiple cores, allowing it to perform multiple computations simultaneously.

Vector Unit: A vector unit is a specialized processing unit that is designed to perform mathematical operations on vectors or arrays of data in a single instruction. Vector units can perform a large number of operations in parallel, making them ideal for applications that involve large amounts of data such as scientific simulations, digital signal processing, and image processing.

The key differences between vector units, threads, and cores are their functions and capabilities. Cores are physical processing units that can execute instructions independently, while threads are units of execution within a process that can run concurrently with other threads on the same core or other cores. Vector units are specialized processing units that are designed to perform mathematical operations on vectors or arrays of data in a single instruction. Threads are also software constructs, while cores are physical hardware components. Threads rely on the underlying hardware, including the cores, to execute instructions. Multiple threads can be scheduled to run on a single core, allowing the core to perform multiple tasks simultaneously. However, each thread can only execute one instruction at a time.

3.5 SIMD Instructions in C++

As briefly mentioned in section 3.3, SIMD (Single Instruction Multiple Data) is a technique for performing data-level parallelism in a computer system. It involves the use of SIMD registers, which are specialized hardware units that can perform the same operation on multiple data elements simultaneously. SIMD is commonly used in applications that require large-scale data processing, such as multimedia, image processing, and scientific simulations. SIMD registers are designed to store multiple data elements of the same data type in a single register. For example, a 256-bit SIMD register can store 8 single-precision floating-point numbers of 4 bytes each or 4 double precision numbers with 8 bytes each. When an operation is performed on a SIMD register, the same operation is applied to all of the data elements stored in the register simultaneously. The use of SIMD registers allows for highly efficient data-level parallelism. Rather than performing the same operation on each data element individually, which can be

time-consuming and inefficient, the operation can be performed on all of the data elements in the SIMD register simultaneously. This results in a significant increase in processing speed and efficiency. Data-level parallelism using SIMD registers can be achieved using a variety of programming languages and APIs, including SIMD extensions to C and C++, and libraries such as Intel Math Kernel Library (MKL) and OpenCV. These tools provide access to SIMD registers and the ability to write SIMD code that takes advantage of the parallel processing capabilities of the registers. For our course, we will be using Intel's Streaming SIMD Extensions (SSE) intrinsic for the SIMD instructions. More specifically, we will be using a C++ class with all the common standard arithmetic operators overloaded for convenience. This class is implemented in the `P4_F32vec4.h` header file.

```
1    // creating a SIMD 0-vector
2    fvec vec = fvec(0.f);
3    // creating a SIMD vector with specific values
4    fvec vec = fvec(1.4f, 2.1f, 0.11f, 1.f);
5    // reinterpret cast dynamic float array to SIMD vector
6    float* arr = (float*) _mm_malloc(8 * sizeof(float), 16);
7    fvec* vec = reinterpret_cast<fvec*>(arr);
```

Listing 3.1: Examples for usage of `P4_F32vec4.h`.

In some cases, it can be challenging to rethink the implementation of a function when using SIMD. For example, the activation function LeakyReLU that is defined as

$$LeakyReLU(x) = \begin{cases} x & \text{if } x \geq 0, \\ 0.1x & \text{otherwise.} \end{cases} \tag{3.1}$$

Here, the function can easily be implemented by using a loop over all elements (neuron values) and a conditional statement if $x < 0$. Depending on the branch, the output will be set as defined. Using SIMD, however, it is more challenging. Assuming a 128-Bit vector unit, each neuron can keep track of 4 float values when SIMDized. Then, a conditional statement is not possible, since some vector elements could be negative, whereas others are positive or zero. One could use a loop to iterate over the SIMD vector elements, but that would be a sequential activation of elements. What can be done instead is using the tools provided by the SIMD extension libraries. There are several functions available, but an elementwise parallel condition statement is not easy to implement, especially due to branch prediction and pipelining. Instead, we use functions that are not depending on branches and conditions, even if it would be intuitive to make use of them. The concept of masks can sometimes be required in these situations. For example, a mask for all positive and negative values can be created. This can be done by using the sign-function of SIMD vectors, that returns a vector including elements that are either -1 or +1, depending on the sign of the provided input parameter vector's elements. This can be easily done by checking the sign bit of each element in parallel.

```
1      // get positive values incl. 0, all others are 0
2      fvec posMask = (sgn(values) + 1.f) / 2.f;
3      fvec posValues = (posMask * values);
4      // apply a factor of 0.01 to all negative values, all others are 0
5      fvec negMask = 1.0f - posValuesMask;
6      fvec negValues = (negMask * values) * 0.01f;
7      // set activated output values
8      values = posValues + negValues;
```

Listing 3.2: Example of rethinking from an SIMD perspective. Other solutions are possible as well.

Analyzing the code snippet, one could argue that the amount of calculations added due to the creation of masks reduces the speedup gained by SIMD, which is generally true. On the other hand, in this implementation no branch prediction (if $x < 0$) is required and the instructions can be easily calculated without the risk of flushing the pipeline due to a mistakenly predicted branch outcome. Furthermore, considering a loop over all neurons, the values are arbitrary, making a branch prediction with high accuracy difficult. Thus, the runtime with SIMD can still be expected faster.

Chapter 4

Vector Class Library Vc

The Vector Class Library (Vc) is a C++ library designed to ease the use of SIMD (Single Instruction, Multiple Data) instructions. SIMD instructions allow parallel processing of data, significantly enhancing performance in computational tasks. However, directly using SIMD instructions such as __m128 can be cumbersome and error-prone, e.g. when moving to a new architecture where the vector register sizes differ. Vc addresses these issues by providing a high-level, easy-to-use interface while maintaining the performance benefits of SIMD.

4.1 Advantages of Vc over Hard-Coded SIMD Instructions

Vc offers several advantages compared to using hard-coded SIMD instructions:

- Abstraction: Vc abstracts the complexities of SIMD instructions, allowing developers to write clean and maintainable code.
- Portability: Code written with Vc is portable across different architectures and SIMD instruction sets (e.g., SSE, AVX). The library handles the underlying details.
- Performance: Vc optimizes the use of SIMD instructions, often providing performance improvements comparable to hand-tuned assembly code.
- Ease of Use: With Vc, developers can leverage familiar C++ idioms and template programming to write efficient SIMD code without needing deep knowledge of the hardware.

4.2 Vc in the C++26 Standard

The importance and utility of Vc have led to its anticipated inclusion in the C++26 standard. Some functionalities are already available in the standard library under `std::experimental`. However, not all features, such as scatter and gather operations, are currently available in this experimental version (https://github.com/VcDevel/std-simd).

```
#include <experimental/simd>
#include <iostream>
```

```
namespace stdx = std::experimental;

using float_v  = stdx::native_simd<float>;

int main() {
    alignas(16) std::array<float, 4> a_arr = {1.f, 2.f, 3.f, 4.f};
    alignas(16) std::array<float, 4> b_arr = {.1f, .2f, .3f, .4f};
    alignas(16) std::array<float, 4> c_arr;


    float_v& a = reinterpret_cast<float_v&>(a_arr[0]);
    float_v& b = reinterpret_cast<float_v&>(b_arr[0]);
    float_v& c = reinterpret_cast<float_v&>(c_arr[0]);

    c = a + b;

    for (size_t i = 0; i < c.size(); ++i) {
        std::cout << "c[" << i << "] = " << c[i] << std::endl;
    }

    return 0;
}
```

4.3 The VcDevel Library

For full functionality, including scatter and gather operations, the VcDevel library should be used. VcDevel is the foundation of Vc and provides all the advanced features that may not yet be available in the standard library (https://github.com/VcDevel/Vc

```
#include <iostream>
#include <Vc/Vc>

float input[N];
float output[N];

int main() {
    // TODO: fill input/output with values

    unsigned int index[float_v::Size];

    // example: gathering
    float_v values_vec;
    uint_v indices(index);
    values_vec.gather(input, indices)

    // example: masked gathering
```

```
    float_m mask = values_vec < 0.1f;
    float_v values_vec2(Vc::Zero);
    values_vec.gather(input, indices, mask);

    // example: masked scattering
    float_m mask2 = values_vec > .05f;
    values_vec.scatter(output, indices, mask2);

    return 0;
}
```

4.4 Conclusion

The Vc library offers a powerful and efficient way to utilize SIMD instructions in C++. Its abstraction and portability make it an excellent choice for high-performance computing tasks. While some features are available in std::experimental, the full capabilities of Vc can be accessed through the VcDevel library. As Vc becomes more integrated into the C++ standard, it is poised to become a fundamental tool for developers seeking to harness the power of SIMD in their applications.

Chapter 5

OpenMP

OpenMP (Open Multi-Processing) is an application programming interface (API) that enables parallel programming in shared-memory systems. It provides a simple and portable way to parallelize code written in C++, C, and Fortran. OpenMP allows developers to exploit the potential of multicore processors, maximizing performance by distributing workload across multiple processing units.

Parallelizing code can significantly improve the performance of computationally intensive applications. OpenMP simplifies the process of writing parallel programs by providing a set of directives, library routines, and environment variables. It allows developers to focus on identifying parallelizable sections of code and relies on the compiler and runtime system to handle the details of thread management and synchronization.

OpenMP offers several advantages for parallel programming:

- Simplified parallelization: OpenMP provides a set of directives, library routines, and environment variables that make parallel programming more accessible.
- Portability: OpenMP is supported by major compilers, making it easy to write parallel code that can be compiled and executed on different platforms.
- Incremental parallelization: OpenMP allows developers to gradually parallelize code by adding directives selectively, focusing on critical sections.
- Flexibility: OpenMP supports various parallelization models, including loop parallelism, task parallelism, and sections.

5.0.1 Compiler Support

To use OpenMP in C++, you need a compiler that supports OpenMP directives and library routines. Most modern compilers, such as GCC, Clang, and Visual C++, include OpenMP support. Once you have a compatible compiler, you can enable OpenMP by adding appropriate compiler flags, such as ”-fopenmp” for GCC and Clang or ”/openmp” for Visual C++. To use OpenMP in your code, you need to include the ”omp.h” header file. OpenMP supports several directives that are used to parallelize code, such as ”#pragma omp parallel” and ”#pragma omp for” which will be detailed below. These directives are used to define parallel regions and distribute loop iterations among multiple threads.

5.0.2 OpenMP Directives

OpenMP directives are pragmas that allow you to specify parallelism in your code. They guide the compiler in generating parallel code and define how the work should be divided among threads. Some commonly used directives include:

- `#pragma omp parallel`: Specifies that the code block following it should be executed in parallel by a team of threads.
- `#pragma omp for`: Distributes the iterations of a loop among the available threads, executing them concurrently.
- `#pragma omp critical`: Marks a section of code as critical, ensuring that only one thread executes it at a time.
- `#pragma omp barrier`: Inserts a synchronization point, forcing all threads to reach that point before continuing execution.
- `#pragma omp sections`: Divides the code block into sections, each to be executed by a different thread.

A parallel region is a block of code that can be executed by multiple threads in parallel. To create a parallel region, you can use the `#pragma omp parallel` directive. This directive is followed by an open brace '' to start the parallel region and a closing brace '' to end it. All the code within the parallel region will be executed by multiple threads. A simple usage of `#pragma omp parallel` is illustrated below:

```
#include <iostream>
#include <omp.h>

int main() {
        int num_threads;

        #pragma omp parallel
        {
                #pragma omp single
                {
                        num_threads = omp_get_num_threads();
                }
        }

        std::cout << "Number of threads: " << num_threads << std::endl;

        return 0;
}
```

In the above example, the code within the parallel region will be executed by multiple threads. The ”omp_get_thread_num()” function returns the ID of the current thread, allowing you to identify which thread is executing the code.

Another useful directive is the `#pragma omp for` directive to parallelize loops. By adding this directive before a loop, you can distribute the iterations of the loop among multiple threads. OpenMP automatically divides the loop iterations and assigns them to different threads for parallel execution.

```
#include <iostream>
#include <omp.h>

int main() {
        const int N = 10;
        int sum = 0;

        #pragma omp parallel for
        for (int i = 0; i < N; ++i) {
                #pragma omp atomic
                sum += i;
        }

        std::cout << "Sum: " << sum << std::endl;
        return 0;
}
```

In the above example, the loop iterations are divided among the available threads, and each thread computes a part of the sum. The `#pragma omp atomic` directive ensures that the updates to the shared variable sum are performed atomically to avoid race conditions.

5.0.3 OpenMP Library Routines

OpenMP provides a set of library routines that allow you to control and query aspects of parallel execution. These routines can be used to manage threads, obtain information about the execution environment, and control thread synchronization. Some commonly used library routines include:

- `omp_get_num_threads()`: Returns the number of threads in the current parallel region.
- `omp_get_thread_num()`: Returns the thread number of the calling thread.
- `omp_set_num_threads()`: Sets the number of threads to be used in subsequent parallel regions.
- `omp_get_wtime()`: Returns the wall-clock time in seconds.

5.1 Data Sharing and Synchronization

When parallelizing code with OpenMP, it is crucial to consider data sharing and synchronization. By default, variables defined outside a parallel region are shared among all threads, which can lead to race conditions and data inconsistencies. OpenMP provides mechanisms to control

data sharing, such as private variables, shared variables, and reduction clauses. Additionally, synchronization constructs like barriers and critical sections help coordinate thread execution and prevent race conditions.

5.2 Performance Considerations

While OpenMP simplifies parallel programming, it's essential to consider performance implications. Load balancing, data dependencies, and overhead due to thread synchronization can affect the scalability and efficiency of parallel code. Profiling tools and performance analysis can help identify bottlenecks and optimize parallel implementations.

5.3 Summary

This chapter introduced OpenMP, a powerful API for parallel programming in shared-memory systems. We discussed its benefits, directives, library routines, data sharing, synchronization, and performance considerations. OpenMP simplifies the process of parallelization by providing a portable and straightforward approach to exploit multicore processors.

Chapter 6

Intel's Thread Building Blocks (TBB) for High-Performance Computing

6.1 Introduction

Welcome to this comprehensive exploration of Intel's Thread Building Blocks (TBB) within the realm of high-performance computing. In the dynamic landscape of modern computing, the ability to leverage parallelism is paramount, and TBB provides a robust solution for tapping into the potential of multi-core processors. This chapter aims to delve into the intricacies of TBB, elucidate its key features, and demonstrate how it can be effectively employed to craft efficient and scalable parallel code.

6.2 Understanding TBB Basics

6.2.1 Parallelism with TBB

TBB simplifies the implementation of parallelism in C++ through a higher-level abstraction for parallel algorithms. The simplicity is best illustrated through an example: consider a scenario where we want to parallelize the computation of the square root of elements in an array.

```
#include <iostream>
#include <tbb/tbb.h>

void ParallelSquareRoot(float* data, size_t size) {
    tbb::parallel_for(size_t(0), size, [&](size_t i) {
        data[i] = std::sqrt(data[i]);
    });
}

int main() {
    const size_t dataSize = 10000;
    float data[dataSize];

```

```
    // Initialize data (omitted for brevity)

    ParallelSquareRoot(data, dataSize);

    // Process the results (omitted for brevity)

    return 0;
}
```

This concise piece of code showcases TBB's `parallel_for` construct, allowing seamless parallelization without the need for intricate threading management.

6.2.2 TBB Task Scheduler

The TBB task scheduler is a critical component that dynamically distributes tasks among threads, optimizing load balancing for improved scalability and performance. Unlike traditional approaches that require manual assignment of tasks to threads, TBB's task scheduler intelligently adapts to the available resources.

Task scheduling is a complex endeavor, and TBB simplifies this process by employing work-stealing techniques. Threads that finish their tasks early can "steal" tasks from other threads, ensuring a balanced workload distribution. This approach is particularly advantageous in scenarios where tasks have varying levels of computational intensity.

TBB's task scheduler operates on the principle of "Divide and Conquer," breaking down larger tasks into smaller, more manageable units. This granularity facilitates efficient load balancing, making it well-suited for applications with irregular workloads.

6.2.3 Parallel Containers

Parallel containers provided by TBB, such as `tbb::concurrent_vector` and `tbb::concurrent_hash_map`, extend the advantages of parallelism to data structures. These containers are designed for concurrent access, eliminating the need for explicit locks and offering a seamless solution for managing shared data in parallel execution environments.

Consider a scenario where multiple threads need to update elements in a shared vector concurrently. Without proper synchronization mechanisms, data corruption and race conditions can occur. TBB's `concurrent_vector` addresses this challenge by providing thread-safe operations on the vector, ensuring consistent and reliable results.

The implementation of these parallel containers emphasizes the importance of thread safety and efficient synchronization. By utilizing lock-free and fine-grained synchronization techniques, TBB minimizes contention among threads, resulting in optimal performance in scenarios with high levels of parallelism.

6.3 Advanced Features of TBB

6.3.1 Flow Graphs

TBB's flow graphs provide a versatile programming model for expressing parallel algorithms in a graph-based fashion. Each node in the graph represents a computational task, and edges define dependencies between tasks. This abstraction simplifies the development of complex parallel workflows, allowing developers to express intricate dependencies with ease.

Consider a scenario where a computational pipeline involves multiple stages with interdependencies. TBB's flow graphs allow the representation of this pipeline as a graph, where each node corresponds to a stage, and edges signify data flow between stages. This high-level representation enhances code readability and facilitates the identification of parallelism opportunities within the pipeline.

Flow graphs in TBB support both static and dynamic graphs. In a static graph, the structure of the graph is defined at compile-time, offering efficiency in scenarios with known dependencies. On the other hand, dynamic graphs allow runtime modification of the graph structure, providing flexibility for applications with varying task dependencies.

The ability to express parallel algorithms as flow graphs is particularly advantageous in scientific computing, image processing, and other domains with intricate task dependencies. TBB's flow graphs empower developers to design and implement parallel workflows that align with the natural structure of their applications.

6.3.2 Data Parallelism

TBB's support for data parallelism is exemplified through parallel algorithms like `tbb::parallel_for` and `tbb::parallel_reduce`. These algorithms enable developers to express parallelism at a higher level, promoting code readability and maintainability.

Consider the example of parallelizing a computation-intensive loop using `tbb::parallel_for`. In traditional programming paradigms, manually distributing loop iterations among threads can be error-prone and lead to load imbalance. TBB's `parallel_for` alleviates this burden by automatically partitioning the loop iterations among available threads, ensuring efficient parallel execution.

```
#include <iostream>
#include <tbb/tbb.h>

void ParallelSquare(float* data, size_t size) {
    tbb::parallel_for(size_t(0), size, [&](size_t i) {
        data[i] = data[i] * data[i];
    });
}

int main() {
    const size_t dataSize = 10000;
    float data[dataSize];

    // Initialize data (omitted for brevity)

```

```
    ParallelSquare(data, dataSize);

    // Process the results (omitted for brevity)

    return 0;
}
```

In this example, the `ParallelSquare` function squares each element in the array concurrently. TBB handles the task distribution and load balancing behind the scenes, allowing developers to focus on the algorithmic aspects of their code.

6.3.3 Scalability

TBB's scalability is a crucial aspect of its design, allowing applications to efficiently utilize resources on both small and large multi-core systems. Achieving scalability in parallel computing involves adapting to varying hardware configurations and efficiently distributing work among available processors.

TBB achieves scalability through several key mechanisms. First and foremost, the dynamic task scheduler adapts to the number of available cores, ensuring that tasks are distributed optimally. This adaptability is particularly important in modern computing environments where the number of cores can vary widely, from laptops with dual-core processors to high-performance servers with dozens of cores.

The concept of scalability also extends to TBB's parallel algorithms and containers. For example, when using `tbb::parallel_for` on a loop, the workload is automatically divided among available threads, ensuring that the overall computation scales with the number of cores. Similarly, parallel containers like `tbb::concurrent_vector` exhibit scalable behavior in scenarios with concurrent access.

Consider a scientific simulation running on a supercomputer with hundreds of cores. TBB's scalability allows the simulation to efficiently utilize the computational resources, resulting in faster execution times and the ability to handle larger problem sizes.

Scalability is a critical consideration in the design of parallel algorithms and libraries, especially in the context of high-performance computing. TBB's emphasis on scalability makes it a versatile tool for developers working on applications that demand optimal performance across a spectrum of multi-core configurations.

6.4 Usefulness in High-Performance Computing

6.4.1 Efficient Utilization of Multi-Core Processors

In high-performance computing, where the ability to efficiently utilize multi-core processors is paramount, TBB proves to be a valuable asset. Traditional approaches to parallel programming often involve intricate thread management and synchronization, adding complexity to the development process.

TBB abstracts away the complexities of managing threads and their synchronization, allowing developers to focus on the algorithmic aspects of their code. The example in Listing **??** demonstrates how TBB's `parallel_for` simplifies the parallelization of a computation-intensive task, in this case, calculating the square root of elements in an array.

Efficient utilization of multi-core processors is not only about dividing work among threads but also about load balancing. TBB's task scheduler dynamically adapts to the available resources, ensuring that tasks are distributed evenly among threads. This load balancing mechanism is crucial in scenarios where the computational workload may vary among different parts of the input data.

Consider an application in computational finance that involves parallel valuation of financial instruments. TBB's task scheduler would dynamically distribute the valuation tasks among available cores, ensuring that the workload is balanced, and the overall computation is completed efficiently.

By providing a higher-level abstraction for parallelism, TBB allows developers to write code that scales across a range of multi-core processors, from desktop machines to high-performance computing clusters. This adaptability is particularly valuable as the landscape of computing continues to evolve with advancements in hardware architectures.

6.4.2 Reduction of Bottlenecks

Efficient parallelism is often hindered by bottlenecks—points in the code where the performance is limited by the slowest component. TBB's task scheduler and parallel algorithms play a crucial role in reducing bottlenecks by efficiently distributing tasks among available threads.

Consider a scenario where an application involves processing a large dataset, and different threads are responsible for analyzing different portions of the data. Without proper load balancing, some threads may finish their tasks quickly, while others may be stuck with more computationally intensive work. This imbalance can lead to bottlenecks and degrade overall performance.

TBB's dynamic task scheduler tackles this challenge by intelligently redistributing tasks among threads. Threads with lighter workloads can "steal" tasks from those with heavier workloads, ensuring a more even distribution of computational work. This adaptability is especially beneficial in scenarios where the workload can vary dynamically.

In the context of scientific simulations, where different regions of a simulation domain may have varying computational requirements, TBB's load balancing capabilities become crucial. By dynamically adjusting to the workload, TBB helps in minimizing idle time and maximizing overall computational throughput.

Reducing bottlenecks is not only about efficient task distribution but also about minimizing contention for shared resources. TBB's parallel containers, such as `tbb::concurrent_vector`, provide thread-safe access to shared data structures, minimizing contention and enhancing overall parallel performance.

The reduction of bottlenecks is a key consideration in high-performance computing, where the efficient use of computational resources is essential. TBB's mechanisms for load balancing and contention management contribute significantly to the alleviation of bottlenecks, enabling applications to achieve optimal performance.

6.4.3 Simplified Parallelism

One of the distinctive features of TBB is its ability to simplify the development of parallel code. The high-level abstractions and parallel algorithms provided by TBB allow developers to express parallelism in a more intuitive and readable manner.

Consider the parallel square root example in Listing **??**. In a traditional multithreading approach, developers would need to manage threads explicitly, handle synchronization, and divide the work among threads manually. TBB abstracts away these low-level details, allowing developers to express the parallelism through a simple and familiar loop construct.

```
#include <iostream>
#include <cmath>
#include <thread>
#include <vector>

void ParallelSquareRoot(float* data, size_t size) {
    const size_t numThreads = std::thread::hardware_concurrency();
    std::vector<std::thread> threads;

    const size_t chunkSize = size / numThreads;

    for (size_t i = 0; i < numThreads; ++i) {
        threads.emplace_back([&, i]() {
            const size_t start = i * chunkSize;
            const size_t end = (i == numThreads - 1) ? size : (i + 1) *
            ↪  chunkSize;

            for (size_t j = start; j < end; ++j) {
                data[j] = std::sqrt(data[j]);
            }
        });
    }

    for (auto& thread : threads) {
        thread.join();
    }
}

int main() {
    const size_t dataSize = 10000;
    float data[dataSize];

    // Initialize data (omitted for brevity)

    ParallelSquareRoot(data, dataSize);

    // Process the results (omitted for brevity)

```

```
    return 0;
}
```

Listing 6.4.3 illustrates how the same task might be approached with traditional multithreading. This code introduces explicit thread management, manual task partitioning, and synchronization, making it more error-prone and less readable compared to the TBB version.

TBB's high-level abstractions not only simplify the development process but also contribute to code maintainability. Parallel algorithms, such as `tbb::parallel_for`, encapsulate common parallel patterns, reducing the likelihood of errors associated with manual thread management.

Consider a scenario where a developer needs to parallelize a loop with a more complex computation, such as matrix multiplication. With TBB, the developer can express this parallelism using `tbb::parallel_for`, focusing on the computation logic without delving into the intricacies of parallelization.

```
#include <iostream>
#include <tbb/tbb.h>
#include <vector>

void ParallelMatrixMultiply(const std::vector<std::vector<double>>& A,
                            const std::vector<std::vector<double>>& B,
                            std::vector<std::vector<double>>& C) {
    const size_t rows = A.size();
    const size_t cols = B[0].
    const size_t rows = A.size();
    const size_t cols = B[0].size();
    const size_t innerDim = B.size();

    tbb::parallel_for(size_t(0), rows, [&](size_t i) {
        for (size_t j = 0; j < cols; ++j) {
            C[i][j] = 0.0;
            for (size_t k = 0; k < innerDim; ++k) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    });
}

int main() {
    const size_t matrixSize = 100;
    std::vector<std::vector<double>> matrixA(matrixSize,
    ↪  std::vector<double>(matrixSize));
    std::vector<std::vector<double>> matrixB(matrixSize,
    ↪  std::vector<double>(matrixSize));
    std::vector<std::vector<double>> resultMatrix(matrixSize,
    ↪  std::vector<double>(matrixSize));

    // Initialize matrices (omitted for brevity)
```

```
31
32      ParallelMatrixMultiply(matrixA, matrixB, resultMatrix);
33
34      // Process the results (omitted for brevity)
35
36      return 0;
37  }
```

Listing 6.4.3 demonstrates how matrix multiplication, a more complex computation, can be parallelized using TBB's `parallel_for`. This approach allows developers to focus on the essential logic of the computation while TBB handles the parallel execution details, resulting in more concise and readable code.

TBB's support for parallel constructs extends beyond loops and matrix multiplication. It provides a rich set of features for expressing parallelism in various scenarios, making it a versatile tool for high-performance computing tasks.

6.4.4 Flow Graphs

Beyond basic parallel constructs, TBB introduces the concept of flow graphs, a powerful tool for expressing parallel algorithms in a graph-based model. Flow graphs represent tasks as nodes, connected by edges denoting dependencies. This abstraction is particularly useful for expressing complex parallel workflows, enhancing the readability and maintainability of parallel code.

TBB's flow graph framework provides a way to express parallelism at a higher level of abstraction, allowing developers to focus on the logical structure of their algorithms rather than low-level threading details. The ability to model dependencies between tasks in a graph simplifies the expression of parallelism for tasks with complex interdependencies. This makes TBB's flow graph a powerful tool for applications with intricate parallel processing requirements.

Consider a scenario where multiple stages of computation depend on each other. Traditional approaches might involve intricate coordination of threads and synchronization mechanisms. With TBB's flow graphs, developers can model these dependencies explicitly, making the parallel workflow more understandable and maintainable.

```
1   #include <iostream>
2   #include <tbb/tbb.h>
3
4   void ParallelWorkflow(const std::vector<double>& input,
5                         std::vector<double>& output) {
6       tbb::flow::graph g;
7
8       // Define nodes and edges for the flow graph (omitted for brevity)
9
10      tbb::flow::make_edge(input_node, processing_node1);
11      tbb::flow::make_edge(processing_node1, processing_node2);
12      tbb::flow::make_edge(processing_node2, output_node);
13
14      g.wait_for_all();
```

```
}

int main() {
    const size_t dataSize = 1000;
    std::vector<double> inputData(dataSize);
    std::vector<double> outputData(dataSize);

    // Initialize input data (omitted for brevity)

    ParallelWorkflow(inputData, outputData);

    // Process the results (omitted for brevity)

    return 0;
}
```

Listing 6.4.4 outlines a simplified example of using TBB's flow graph to express a parallel workflow. In this scenario, tasks are represented as nodes, and dependencies between them are explicitly defined. TBB's flow graph framework manages the parallel execution of these tasks, providing a clear and concise representation of the parallel workflow.

TBB's flow graphs are particularly advantageous in scenarios where the parallelism involves multiple stages of computation with dependencies, such as data processing pipelines.

6.4.5 Data Parallelism

TBB's support for data parallelism is exemplified through parallel algorithms such as `tbb::parallel_for` and `tbb::parallel_reduce`. These algorithms provide a higher-level interface for expressing parallelism, allowing developers to focus on the algorithmic aspects rather than low-level thread management. The expressiveness of TBB's data parallelism support makes it an attractive choice for high-performance computing applications, where readability and ease of maintenance are critical factors.

Moreover, TBB's data parallelism features are designed to handle a variety of data structures, from simple arrays to complex containers, making it versatile for different types of applications. This flexibility allows developers to apply TBB's data parallelism features to a wide range of computational tasks, from numerical simulations to data processing pipelines. The ability to parallelize data manipulations with minimal code modifications contributes to the efficiency and scalability of applications developed with TBB.

6.4.6 Scalability

TBB's scalability is a defining feature, adapting dynamically to the underlying hardware. This adaptability is crucial for high-performance computing systems with varying numbers of cores. TBB's ability to efficiently utilize resources across different architectures ensures optimal performance, making it a versatile tool for a wide range of parallel computing scenarios. Whether dealing with a small cluster or a large supercomputer, TBB's design principles promote efficient resource utilization.

Furthermore, TBB's scalability is not only limited to the number of cores but also extends to different types of parallel workloads. Applications with coarse-grained parallelism, where tasks are large and independent, can benefit from TBB's scalability just as much as applications with fine-grained parallelism, where tasks are small and interdependent. This flexibility makes TBB well-suited for a diverse set of parallel computing challenges.

6.5 Usefulness in High-Performance Computing

6.5.1 Efficient Utilization of Multi-Core Processors

In high-performance computing, TBB proves invaluable by abstracting the complexity of managing threads and their synchronization. This abstraction enables developers to concentrate on the algorithmic aspects of their code, a significant advantage when dealing with intricate computational tasks. TBB's efficient utilization of multi-core processors is pivotal for achieving optimal performance in computationally intensive applications.

Moreover, TBB's support for dynamic load balancing plays a crucial role in efficiently distributing computational tasks across available cores. This adaptability ensures that each core is utilized to its full potential, preventing situations where some cores are idle while others are overloaded. In the world of high-performance computing, where computational resources are a precious commodity, TBB's ability to maximize multi-core processor usage is a key factor in its usefulness.

6.5.2 Reduction of Bottlenecks

TBB's task scheduler plays a crucial role in reducing bottlenecks in parallel applications. By intelligently distributing tasks among available threads, TBB minimizes contention and ensures a balanced workload. In high-performance computing scenarios, where uneven workloads can significantly impact overall performance, TBB's load balancing capabilities become instrumental for achieving scalability.

Furthermore, TBB's ability to adapt to varying workloads and dynamically allocate resources based on the available cores contributes to bottleneck reduction. The task scheduler's efficiency in managing tasks with different execution times prevents situations where some threads finish early and remain idle while others are still processing tasks. This adaptability enhances the overall efficiency of parallel applications, making TBB well-suited for high-performance computing environments with unpredictable workloads.

6.5.3 Simplified Parallelism

TBB's high-level abstractions and parallel algorithms simplify the development of parallel code. This simplicity is particularly significant in high-performance computing, where complex parallel tasks are commonplace. TBB allows developers to express parallelism more intuitively, resulting in faster development cycles and code that is easier to maintain.

Additionally, TBB's support for parallel algorithms, such as parallel reductions and parallel scans, abstracts away the intricacies of manual thread management. Developers can leverage these algorithms to express parallelism in a concise and readable manner, focusing on the algorithm's

logic rather than the nitty-gritty details of thread coordination. This abstraction is crucial in high-performance computing, where code readability and maintainability are essential for long-term success.

The reduction in complexity offered by TBB extends beyond individual parallel algorithms. TBB's task scheduler dynamically adapts to the available hardware, eliminating the need for developers to fine-tune thread counts manually. This adaptive approach not only simplifies the development process but also contributes to the portability of parallel code across different hardware configurations.

In addition to simplifying parallelism, TBB facilitates the creation of scalable and responsive applications. By intelligently managing threads and tasks, TBB minimizes contention and efficiently utilizes available resources. This results in applications that can seamlessly scale with increasing computational demands, making TBB a valuable asset in the toolkit of developers working on high-performance computing projects.

6.6 Real-World Application Examples

6.6.1 Example 1: Image Processing

Consider an image processing application that applies complex filters to a large dataset of images. TBB can be utilized to parallelize the filtering process, distributing the workload across available cores and significantly reducing the overall processing time. TBB's ability to efficiently process large volumes of data in parallel makes it an excellent choice for image processing tasks in high-performance computing environments.

Moreover, TBB's support for parallel loops allows developers to easily parallelize image processing tasks without the need for intricate thread management. By expressing the filtering operation as a parallel for loop, developers can leverage TBB's task scheduler to automatically distribute the work among available cores. This not only accelerates the image processing task but also simplifies the parallelization process, making TBB a valuable tool for image processing applications in high-performance computing.

6.6.2 Example 2: Scientific Simulations

In scientific simulations, such as those used in computational physics or chemistry, TBB can be employed to parallelize complex numerical computations. This parallelization ensures faster simulations and allows researchers to explore larger parameter spaces. TBB's adaptability to varying hardware configurations makes it a reliable choice for accelerating scientific simulations, contributing to advancements in various research domains.

Furthermore, TBB's support for parallel reductions is particularly beneficial in scientific simulations where aggregating results from parallel computations is common. Whether simulating physical phenomena or exploring chemical reactions, TBB's parallel reduction capabilities enable efficient data aggregation across multiple threads, enhancing the overall performance of scientific simulations. The ability to parallelize both computation and data aggregation makes TBB a valuable asset in the realm of high-performance scientific computing.

6.7 Challenges and Considerations

6.7.1 Thread Safety

Although TBB simplifies concurrent programming, developers must ensure thread safety in their applications. Understanding the thread safety requirements of TBB components and adhering to best practices is essential to prevent data corruption and race conditions. While TBB handles many aspects of thread safety internally, developers should remain vigilant in managing shared resources appropriately.

Moreover, TBB's support for concurrent containers introduces an additional layer of complexity concerning thread safety. While these containers are designed for concurrent access, developers should be mindful of potential race conditions and ensure proper synchronization when accessing shared data structures. Adhering to thread safety principles is crucial to avoid unexpected behavior and maintain the integrity of parallel applications.

6.7.2 Overhead and Scalability Limits

While TBB excels in many scenarios, there may be cases where the overhead of task creation and scheduling outweighs the benefits of parallelism, especially in fine-grained parallelism. Developers should carefully analyze their application's characteristics and performance requirements to determine the optimal use of TBB. Understanding the potential scalability limits and overhead associated with TBB is crucial for making informed decisions in high-performance computing projects.

Furthermore, TBB's performance characteristics may vary based on the nature of the workload and the specific algorithms used. Fine-tuning TBB parameters, such as the granularity of tasks and the number of threads, may be necessary to achieve optimal performance in certain scenarios. Developers should conduct thorough performance testing and profiling to identify potential bottlenecks and ensure that TBB's benefits are maximized.

6.7.3 Learning Curve

For developers new to parallel programming or TBB, there may be a learning curve associated with understanding its concepts and optimal usage. However, the benefits gained from utilizing TBB often outweigh the initial investment in learning. The availability of comprehensive documentation, tutorials, and community support can significantly ease the learning process for developers venturing into parallel programming with TBB.

Additionally, TBB's higher-level abstractions and parallel constructs are designed to simplify parallel programming tasks. While there may be an initial learning curve, the intuitive nature of TBB's interface contributes to faster adoption and proficiency. Developers can leverage TBB's documentation, online resources, and community forums to accelerate the learning process and unlock the full potential of parallel programming with TBB.

6.8 Conclusion

In conclusion, Intel's Thread Building Blocks emerges as a cornerstone in the toolkit of developers aiming to unlock the full potential of multi-core processors. Its high-level abstractions, dynamic task scheduler, and support for various parallel patterns make it an invaluable asset in the world of high-performance computing. As we navigate the challenges of scalability and parallelism in modern computing, TBB stands out as an effective and versatile solution for crafting efficient and scalable parallel code.

Thank you for joining today's lecture on Intel's Thread Building Blocks. I encourage you to explore TBB further, experiment with its features, and integrate it into your parallel programming arsenal. Happy coding!

6.9 Other Code Examples

6.9.1 Scalar Implementation

Example: Sum of Array Elements

```
int sumArray(const std::vector<int>& arr) {
    int sum = 0;
    for (int value : arr) {
        sum += value;
    }
    return sum;
}
```

6.9.2 Parallel Implementation Using TBB

Example: Parallel Sum of Array Elements

```
#include <tbb/parallel_reduce.h>
#include <tbb/blocked_range.h>

int parallelSumArray(const std::vector<int>& arr) {
    return tbb::parallel_reduce(
        tbb::blocked_range<std::vector<int>::const_iterator>(arr.begin(),
        ↪  arr.end()),
        0,
        [](const tbb::blocked_range<std::vector<int>::const_iterator>& range,
        ↪  int init) {
        for (auto it = range.begin(); it != range.end(); ++it) {
        init += *it;
        }
        return init;
        },
```

```
14          std::plus<int>()
15      );
16  }
```

Chapter 7

Open Computing Language (OpenCL)

OpenCL (Open Computing Language) is an open, royalty-free standard for cross-platform, parallel programming of diverse accelerators found everywhere from supercomputers to mobile devices. OpenCL was created and is managed by the Khronos Group. By enabling applications to tap into the power of accelerated parallel programming, OpenCL greatly improves the speed and responsiveness of a wide range of applications, engines and libraries - including professional creative tools, scientific and medical software, vision processing, and neural network training and inference.

7.1 Introduction

OpenCL is a programming framework and runtime that enables a programmer to create small programs, called kernel programs (or kernels), that can be compiled and executed, in parallel, across any processor in a heterogeneous system. The processors can be any mix of different CPUs, GPUs, DSPs, FPGAs or Tensor Processors.

OpenCL is a low-level programming framework so the programmer has direct, explicit control over where and when kernels are run, how the memory they use is allocated and how the compute devices and host CPU synchronize their operations to ensure that data and computed results flow correctly - even when the host and compute kernels are running in parallel. The most commonly used language for programming the kernels that are compiled and executed across the available parallel processors is called OpenCL C. OpenCL C is based on C99 and is defined as part of the OpenCL specification.

An OpenCL application is split into host code and device kernel code. Host code is typically written using a general programming language such as C or C++ and compiled by a conventional compiler for execution on the host CPU. Device kernels that are written in OpenCL C, which is based on C99, can be ingested and compiled by the OpenCL driver during execution of an application using runtime OpenCL API calls. This is called online compilation and is supported by all OpenCL drivers. OpenCL C is a subset of ISO C99 with language extensions for parallelism, well-defined numerical accuracy (IEEE 754 rounding with specified max error) and a rich set of built-in functions including cross, dot, sin, cos, pow, log etc.

7.2 OpenCL Programming Model

To understand how to program OpenCL in more detail let's consider the Platform, Execution and Memory Models. The three models interact and define OpenCL's essential operation.

7.2.1 Platform Model

The OpenCL Platform Model describes how OpenCL understands the compute resources in a system to be topologically connected.

A host is connected to one or more OpenCL compute devices. Each compute device is collection of one or more compute units where each compute unit is composed of one or more processing elements. Processing elements execute code with SIMD (Single Instruction Multiple Data) or SPMD (Single Program Multiple Data) parallelism. For example, a compute device could be a GPU. Compute units would then correspond to the streaming multiprocessors (SMs) inside the GPU, and processing elements correspond to individual streaming processors (SPs) inside each SM. Processors typically group processing elements into compute units for implementation efficiency.

7.2.2 Execution Model

OpenCL's `clEnqueueNDRangeKernel` command enables a single kernel program to be initiated to operate in parallel across an N-dimensional data structure. Using a two-dimensional image as a example, the size of the image would be the NDRange, and each pixel is called a work-item that a copy of kernel running on a single processing element will operate on.

As we saw in the Platform Model section above, it is common for processors to group processing elements into compute units for execution efficiency. Therefore, when using the `clEnqueueNDRangeKernel` command, the program specifies a work-group size that represents groups of individual work-items in an NDRange that can be accommodated on a compute unit. Work-items in the same work-group are able to share local memory, synchronize more easily using work-group barriers, and cooperate more efficiently using work-group functions such as `async_work_group_copy` that are not available between work-items in separate work-groups.

7.2.3 Memory Model

7.3 OpenCL Functions

This section provides documentation for the OpenCL functions. Each function is explained along with its parameters.

- `clGetPlatformIDs`
- `clGetDeviceIDs`
- `clCreateContext`

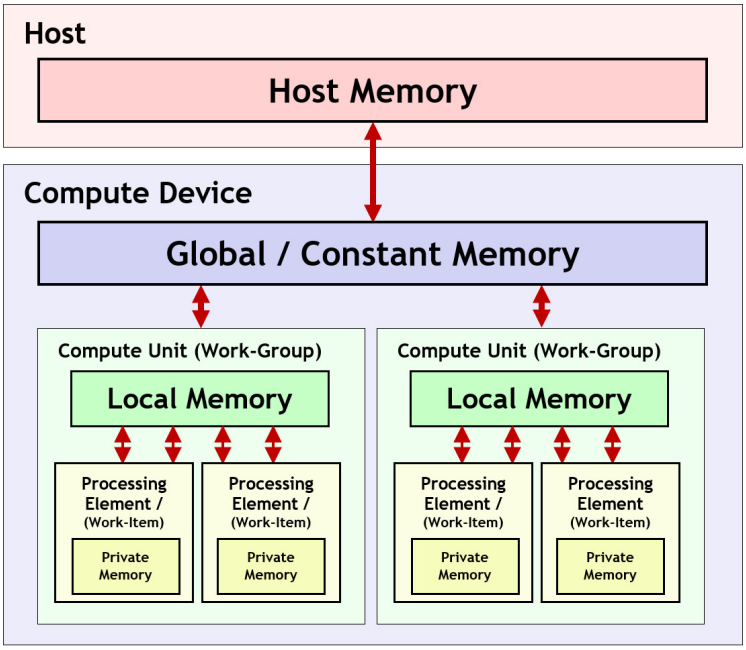

Figure 7.1: Memory Model in OpenCL

- `clCreateCommandQueue`
- `clCreateBuffer`
- `clCreateProgramWithSource`
- `clBuildProgram`
- `clCreateKernel`
- `clSetKernelArg`
- `clEnqueueNDRangeKernel`
- `clEnqueueReadBuffer`
- `clReleaseMemObject`
- `clReleaseKernel`
- `clReleaseProgram`
- `clReleaseCommandQueue`
- `clReleaseContext`

7.3.1 `clGetPlatformIDs`

Retrieves the list of OpenCL platforms available.

```
cl_int clGetPlatformIDs(
    cl_uint num_entries,
    cl_platform_id *platforms,
    cl_uint *num_platforms
);
```

num_entries The maximum number of `cl_platform_id` entries that can be added to the `platforms` array. This sets the size of the array you are passing to hold the platform IDs.

platforms An array to receive the OpenCL platform IDs. If `platforms` is `NULL`, this argument is ignored.

num_platforms A pointer to the number of OpenCL platforms available. If `num_platforms` is `NULL`, this argument is ignored.

7.3.2 clGetDeviceIDs

Retrieves the list of devices available on a specified platform.

```
cl_int clGetDeviceIDs(
    cl_platform_id platform,
    cl_device_type device_type,
    cl_uint num_entries,
    cl_device_id *devices,
    cl_uint *num_devices
);
```

platform The platform ID to query for available devices.

device_type The type of device to query. Examples include `CL_DEVICE_TYPE_GPU` for GPU devices and `CL_DEVICE_TYPE_CPU` for CPU devices.

num_entries The maximum number of `cl_device_id` entries that can be added to the `devices` array. This sets the size of the array you are passing to hold the device IDs.

devices An array to receive the OpenCL device IDs. If `devices` is `NULL`, this argument is ignored.

num_devices A pointer to the number of OpenCL devices available. If `num_devices` is `NULL`, this argument is ignored.

7.3.3 clCreateContext

Creates an OpenCL context.

```
cl_context clCreateContext(
    const cl_context_properties *properties,
    cl_uint num_devices,
    const cl_device_id *devices,
    void (CL_CALLBACK *pfn_notify)(
        const char *errinfo,
        const void *private_info,
        size_t cb,
        void *user_data
    ),
    void *user_data,
    cl_int *errcode_ret
);
```

properties A list of context properties and their values. This is typically used for additional settings and is often `NULL`.

num_devices The number of devices specified in the `devices` array.

devices An array of devices that will be part of the OpenCL context.

pfn_notify A callback function that will be called if any error occurs during the execution of OpenCL commands in this context. This can be `NULL` if no callback is desired.

user_data User-specified data that will be passed to the callback function. This can be `NULL` if no callback is used.

errcode_ret A pointer to an integer where the error code will be stored. This can be `NULL` if the error code is not needed.

7.3.4 clCreateCommandQueue

Creates a command queue on a specific device.

```
cl_command_queue clCreateCommandQueue(
    cl_context context,
    cl_device_id device,
    cl_command_queue_properties properties,
    cl_int *errcode_ret
);
```

context A valid OpenCL context that the command queue will be created in.

device A device associated with the context on which the commands will be executed.

properties A bitfield to specify properties for the command queue, such as `CL_QUEUE_PROFILING_ENABLE` for profiling information.

errcode_ret A pointer to an integer where the error code will be stored. This can be `NULL` if the error code is not needed.

7.3.5 `clCreateBuffer`

Creates a buffer object.

```
cl_mem clCreateBuffer(
    cl_context context,
    cl_mem_flags flags,
    size_t size,
    void *host_ptr,
    cl_int *errcode_ret
);
```

`context` A valid OpenCL context.

`flags` A bitfield that specifies allocation and usage information about the buffer. For example, `CL_MEM_READ_WRITE` specifies that the buffer will be read from and written to.

`size` The size in bytes of the buffer to be allocated.

`host_ptr` A pointer to the buffer data that may already be allocated by the application. This can be `NULL` if no data is to be copied.

`errcode_ret` A pointer to an integer where the error code will be stored. This can be `NULL` if the error code is not needed.

7.3.6 `clCreateProgramWithSource`

Creates a program object for a context and loads the source code.

```
cl_program clCreateProgramWithSource(
    cl_context context,
    cl_uint count,
    const char **strings,
    const size_t *lengths,
    cl_int *errcode_ret
);
```

`context` A valid OpenCL context.

`count` The number of strings in the `strings` array.

`strings` An array of `count` pointers to character strings that make up the source code.

`lengths` An array with the number of characters in each string. If `NULL`, the strings are assumed to be null-terminated.

`errcode_ret` A pointer to an integer where the error code will be stored. This can be `NULL` if the error code is not needed.

7.3.7 clBuildProgram

Builds (compiles and links) a program executable from the program source or binary.

```
cl_int clBuildProgram(
    cl_program program,
    cl_uint num_devices,
    const cl_device_id *device_list,
    const char *options,
    void (CL_CALLBACK *pfn_notify)(cl_program program, void *user_data),
    void *user_data
);
```

program The program object to be built.

num_devices The number of devices listed in `device_list`.

device_list A list of devices that are in the program's context.

options A string containing build options. These options are passed to the compiler.

pfn_notify A callback function that can be used to report when the program is built. This can be `NULL` if no callback is desired.

user_data User data that is passed to the callback function. This can be `NULL` if no callback is used.

7.3.8 clCreateKernel

Creates a kernel object.

```
cl_kernel clCreateKernel(
    cl_program program,
    const char *kernel_name,
    cl_int *errcode_ret
);
```

program A valid program object that has been successfully built with `clBuildProgram`.

kernel_name A function name in the program declared with the `__kernel` qualifier.

errcode_ret A pointer to an integer where the error code will be stored. This can be `NULL` if the error code is not needed.

7.3.9 `clSetKernelArg`

Sets the argument value for a specific argument of a kernel.

```
cl_int clSetKernelArg(
    cl_kernel kernel,
    cl_uint arg_index,
    size_t arg_size,
    const void *arg_value
);
```

`kernel` A valid kernel object.

`arg_index` The argument index. Arguments are indexed starting from 0.

`arg_size` The size of the argument value.

`arg_value` A pointer to the argument value.

7.3.10 `clEnqueueNDRangeKernel`

Enqueues a command to execute a kernel on a device.

```
cl_int clEnqueueNDRangeKernel(
    cl_command_queue command_queue,
    cl_kernel kernel,
    cl_uint work_dim,
    const size_t *global_work_offset,
    const size_t *global_work_size,
    const size_t *local_work_size,
    cl_uint num_events_in_wait_list,
    const cl_event *event_wait_list,
    cl_event *event
);
```

`command_queue` A valid command queue where the kernel will be queued for execution.

`kernel` A valid kernel object to be executed.

`work_dim` The number of dimensions used to specify the global and local work sizes.

`global_work_offset` A pointer to an array of dimension offsets for the global IDs. This can be `NULL` if no offset is needed.

`global_work_size` A pointer to an array that describes the number of global work-items in each dimension.

`local_work_size` A pointer to an array that describes the number of work-items that make up a work-group. This can be `NULL` if the OpenCL implementation determines the work-group size.

num_events_in_wait_list The number of events in the `event_wait_list`.

event_wait_list A list of events that need to complete before this command can be executed. This can be `NULL` if no event dependencies are needed.

event Returns an event object that identifies this particular kernel execution instance. This can be `NULL` if no event is needed.

7.3.11 clEnqueueReadBuffer

Enqueues a command to read from a buffer object to host memory.

```
cl_int clEnqueueReadBuffer(
    cl_command_queue command_queue,
    cl_mem buffer,
    cl_bool blocking_read,
    size_t offset,
    size_t size,
    void *ptr,
    cl_uint num_events_in_wait_list,
    const cl_event *event_wait_list,
    cl_event *event
);
```

command_queue A valid command queue.

buffer A valid buffer object.

blocking_read A boolean value that indicates if the read operation is blocking or non-blocking. If `CL_TRUE`, the read is blocking.

offset The offset in the buffer object to read from.

size The size in bytes of data being read.

ptr A pointer to the buffer in host memory where the data is to be read into.

num_events_in_wait_list The number of events in the `event_wait_list`.

event_wait_list A list of events that need to complete before this command can be executed. This can be `NULL` if no event dependencies are needed.

event Returns an event object that identifies this particular read instance. This can be `NULL` if no event is needed.

7.3.12 clReleaseMemObject

Decrements the memory object reference count.

```
cl_int clReleaseMemObject(cl_mem memobj);
```

memobj A valid memory object. This function releases the OpenCL memory object and deallocates its resources once the reference count reaches zero.

7.3.13 clReleaseKernel

Decrements the kernel reference count.

```
cl_int clReleaseKernel(cl_kernel kernel);
```

kernel A valid kernel object. This function releases the kernel object and deallocates its resources once the reference count reaches zero.

7.3.14 clReleaseProgram

Decrements the program reference count.

```
cl_int clReleaseProgram(cl_program program);
```

program A valid program object. This function releases the program object and deallocates its resources once the reference count reaches zero.

7.3.15 clReleaseCommandQueue

Decrements the command queue reference count.

```
cl_int clReleaseCommandQueue(cl_command_queue command_queue);
```

command_queue A valid command queue. This function releases the command queue and deallocates its resources once the reference count reaches zero.

7.3.16 clReleaseContext

Decrements the context reference count.

```
cl_int clReleaseContext(cl_context context);
```

context A valid OpenCL context. This function releases the OpenCL context and deallocates its resources once the reference count reaches zero.

Chapter 8

Compute Unified Device Architecture (CUDA)

8.1 System Components and Architecture

Modern heterogeneous computing systems comprise several key hardware components that collaborate to execute high-performance applications. In the context of GPU programming, the primary components include the central processing unit (CPU) and system memory (RAM), as already discussed in previous contexts, as well as the graphics processing unit (GPU) and its dedicated video memory (VRAM).

The CPU continues to act as the main orchestrator of program execution. A more recent responsibility in GPU-accelerated applications is managing data transfers between main memory and peripheral devices such as the GPU. While system RAM provides the CPU with fast, volatile storage for active data and instructions, the GPU maintains its own high-bandwidth memory (VRAM) to allow efficient access to data during massively parallel computations.

The GPU is typically connected via a PCI Express (PCIe) interface on the mainboard. Architecturally, it differs significantly from the CPU: whereas the CPU consists of a small number of complex cores optimized for sequential workloads, the GPU contains thousands of simpler cores optimized for highly parallel workloads. This design makes GPUs particularly well-suited for applications with high data parallelism, such as large-scale numerical simulations and machine learning inference and training.

Each GPU has access to its own VRAM, which holds input data, intermediate results, and output produced during computation. However, communication between the CPU and GPU must traverse the PCIe bus, which, despite being relatively fast, offers significantly lower bandwidth and higher latency compared to on-chip memory or CPU–RAM communication. Consequently, data transfer between host (CPU) and device (GPU) can become a performance bottleneck, especially for applications involving frequent or large data transfers.

All components are interconnected via the system's mainboard, which manages power distribution, connectivity, and communication pathways. Maximizing performance in GPU-accelerated environments requires careful consideration of both computational and data transfer characteristics to minimize bottlenecks and fully exploit the available hardware capabilities.

8.2 GP-GPU Programming

Originally developed for accelerating graphics rendering, graphics processing units (GPUs) have evolved into powerful platforms for general-purpose computing. This paradigm shift, commonly referred to as General-Purpose computing on Graphics Processing Units (GP-GPU), enables developers to harness the massive parallelism of GPUs for a wide range of non-graphical tasks.

GP-GPU programming is particularly well-suited for data-parallel applications that require high-throughput computation. Typical use cases include scientific simulations, deep learning, financial modeling, and real-time data processing.

Programming models such as CUDA (developed by Nvidia) and HIP (developed by AMD) have significantly simplified GPU programming. These modern frameworks provide abstractions that allow developers to define parallel kernels, manage device memory, and coordinate execution between the host and the GPU.

While CUDA and HIP offer efficient, vendor-specific access to GPU hardware, they are inherently tied to their respective ecosystems. In contrast, OpenCL provides a more portable programming model that supports a wide range of hardware platforms, including CPUs, GPUs, and FPGAs. However, this increased portability comes at the cost of greater programming complexity and typically more limited opportunities for low-level performance optimization.

Efficient use of GPU programming frameworks requires a solid understanding of both the underlying hardware (e.g., GPU architecture and memory hierarchy) and the associated programming model (e.g., memory allocation strategies and data transfer mechanisms) in order to achieve high performance and scalability.

8.3 CUDA Workflow

The typical CUDA programming workflow follows a sequence of steps that facilitate data transfer, parallel execution, and synchronization between the host (CPU) and device (GPU). These steps are outlined below:

1. **Initialize data on the CPU:** Allocate and initialize memory on the host. This includes preparing input data structures and any necessary parameters for GPU execution.
2. **Transfer data to the GPU (RAM → VRAM):** Copy input data from host memory (RAM) to device memory (VRAM) using functions such as `cudaMemcpy`. Efficient memory transfer is critical for performance.
3. **Launch the kernel:** Invoke a kernel function that runs on the GPU, specifying the number of threads and thread blocks to be used. Similar to multi-threading on CPU, each thread typically handles a portion of the data in parallel.
4. **(Optional) Run independent work on the CPU:** While the GPU is executing the kernel asynchronously, the CPU can perform tasks that do not depend on the kernel result, improving overall resource utilization.
5. **Synchronize the GPU:** Ensure that the kernel execution is complete before proceeding. This is typically done using `cudaDeviceSynchronize` or by synchronizing specific streams.

6. **Transfer data back to the CPU (VRAM → RAM):** Copy the results of the GPU computation from device memory back to host memory for further processing or output.
7. **Post-process data on the CPU:** Perform any necessary final computations, analysis, or output formatting on the results received from the GPU.

8.4 CUDA Terminology

In this section, the most important keywords in the context of CUDA GPU programming are listed. These terms should be understood to effectively read, write, and discuss CUDA code.

Streaming Multiprocessor (SM). The fundamental hardware execution unit within a GPU. Each SM contains multiple arithmetic units, memory, and schedulers that execute groups of threads. All threads within a block are assigned to the same SM during execution.

Grid. The entire collection of threads launched by a single kernel call. A grid is composed of multiple thread blocks and defines the full execution space of a kernel.

Block. A group of threads that execute in parallel on the same streaming multiprocessor (SM). Threads within a block can cooperate via fast on-chip shared memory and synchronize with one another.

Thread. The smallest unit of execution. Each thread executes the same kernel function but operates on different data. Threads have their own local memory and registers.

Kernel. A device function marked with the `__global__` qualifier, typically launched from the CPU. It defines the operations to be performed in parallel by many threads on the GPU.

Host vs. Device. The *host* refers to the CPU and its main memory (RAM), where the program is started. The *device* refers to the GPU and its memory (VRAM), where parallel computations are offloaded.

Stream. A sequence of commands (such as memory transfers or kernel executions) issued to the GPU. Streams allow overlapping operations, for example, running computations while simultaneously copying data.

8.5 CMake and CUDA

CMake also works with CUDA. Below you will find a small example of an updated CMakeList.txt that will help you to adjust yours for the new exercise.

```
cmake_minimum_required(VERSION 3.27)
project(GameOfLife LANGUAGES CXX CUDA)

add_compile_definitions(USE_CUDA)

set(CMAKE_CXX_STANDARD 17)

set(CMAKE_CXX_FLAGS_RELEASE "${CMAKE_CXX_FLAGS_RELEASE} -O3 -fopenmp")

```

```
include_directories(include)

add_executable(GameOfLife
    GameOfLife.cpp
    src/World.cpp
    src/CLInterface.cpp
    src/CUDAEvolution.cu
)

target_compile_definitions(GameOfLife PRIVATE USE_CUDA)

set_target_properties(GameOfLife PROPERTIES
    CUDA_SEPARABLE_COMPILATION ON
    POSITION_INDEPENDENT_CODE ON
)
```

8.6 Code Examples

In this section you will find several code examples that will help you with solving the exercises.

8.6.1 A Simple Kernel

```
// CPU function
void CPUFunction()
{
  printf("CPU\n");
}

// GPU kernel function
__global__ void GPUFunction()
{
  printf("GPU\n");
}

int main() {
  CPUFunction();
  GPUFunction<<<1, 1>>>();
  cudaDeviceSynchronize();
}
```

8.6.2 Extraction of GPU Information

```
int deviceId;
cudaGetDevice(&deviceId);

```

```
cudaDeviceProp deviceProp;
cudaGetDeviceProperties(&deviceProp, deviceId);

int computeCapabilityMajor = deviceProp.major;
int computeCapabilityMinor = deviceProp.minor;
int multiProcessorCount = deviceProp.multiProcessorCount;
int warpSize = deviceProp.warpSize;
```

8.6.3 Grid-Stride Loops

```
__global__ void doubleElements(int *a, int N)
{
  int i = threadIdx.x + blockIdx.x * blockDim.x;
  int stride = gridDim.x * blockDim.x;
  for (; i < N; i += stride) a[i] *= 2;
}
```

8.6.4 Unified Memory Management

```
int main()
{
  int N = 10000;
  int *a;
  size_t size = N * sizeof(int);

  cudaMallocManaged(&a, size);  // allocation of unified (!) memory

  init(a, N); // e.g. by mapping a[i] = i

  size_t n_threads = 256;
  size_t n_blocks = 32;

  // Note: the size of this grid is 256*32 = 8192 < 10000 (= N).
  doubleElements<<<n_blocks, n_threads>>>(a, N);
  cudaDeviceSynchronize();

  cudaFree(a); // free allocated unified memory
}
```

8.6.5 Manual Memory Management

```
int size = 100'000;

double *src, *dest;
cudaMallocHost(&src, sizeof(double) * size);
cudaMallocHost(&dest, sizeof(double) * size);
```

```

double *d_src, *d_dest;
cudaMalloc(&d_src, sizeof(double) * size);
cudaMalloc(&d_dest, sizeof(double) * size);

cudaMemcpy(d_src, src, sizeof(double) * size, cudaMemcpyHostToDevice);

int n_blocks = 64;
int n_threads = 32;
kernel<<<n_blocks, n_threads>>>(d_src, d_dest, size);

cudaDeviceSynchronize();

cudaMemcpy(dest, d_dest, sizeof(double) * size, cudaMemcpyDeviceToHost);

cudaFree(d_src);
cudaFree(d_dest);

cudaFreeHost(src);
cudaFreeHost(dest);
```

8.6.6 Creating CUDA Streams

```
// Create a stream:
cudaStream_t stream;
cudaStreamCreate(&stream);

// memory allocation / data initialization
...

// launch a kernel into non-default stream:
kernel<<<grid, blocks, 0, stream>>>();

// cleanup
cudaStreamDestroy(stream);
```

8.6.7 Manual Asynchronous Memory Management

```
cudaStream_t stream;
cudaStreamCreate(&stream);

const uint64_t num_entries = 1UL << 26;

uint64_t *data_cpu, *data_gpu;

cudaMallocHost(&data_cpu, sizeof(uint64_t)*num_entries);    // pinned memory

```

```
cudaMalloc(&data_gpu, sizeof(uint64_t)*num_entries);

cudaMemcpyAsync(data_gpu,
                data_cpu,
                sizeof(uint64_t)*num_entries,
                cudaMemcpyHostToDevice,
                stream);

// do something
...

cudaMemcpyAsync(data_cpu,
                data_gpu,
                sizeof(uint64_t)*num_entries,
                cudaMemcpyDeviceToHost,
                stream);

cudaStreamSynchronize(stream); // can be used to wait until data is fully
↪  transferred.
```

8.6.8 Error Handling

It is always recommended to check for errors that can occur during execution of GPU instructions. Therefore, we suggest to implement a function similar to the following one that helps to find errors that happen at different stages of your code.

```
inline cudaError_t checkCuda(cudaError_t result) {
  if (result ≠ cudaSuccess) {
    fprintf(stderr, "Error: %s\n", cudaGetErrorString(result));
    assert(result == cudaSuccess);
  }
  return result;
}
```

If done, one can easily check for errors:

```
int main(...)
{
  // check errors like this:
  cudaError_t err;
  err = cudaMallocManaged(&a, N);
  checkCuda(err);

  doubleElements<<<n_blocks, n_threads>>>(a, N);

  // or check errors like this:
  checkCuda(cudaDeviceSynchronize());

```

```
  // or this:
  checkCuda(cudaGetLastError()); // cudaGetLastError will return the error from
  ↪ above.
}
```

8.7 Multi-GPU Programming

Multi-GPU programming will not be part of this course in general, but this section provides an example if you are interested in the topic itself.

In CUDA, one can simply loop over the devices (GPUs) to distribute the workload.

```
int num_gpus;
cudaGetDeviceCount(&num_gpus);

for (int gpu = 0; gpu < num_gpus; gpu++) {

    cudaSetDevice(gpu);

    // Perform operations for this GPU.
    ...
}
```

In general, it should be considered that the work of different streams can be processed in any order and, therefore, it is crucial to allocate memory and transfer data well in time before the kernels are launched.

While the previous instruction chain can be summarized as:

1. create streams
2. allocate host memory
3. allocate device memory
4. transfer data to device
5. launch kernels
6. synchronize host and device
7. transfer data from device
8. destroy streams
9. free all memory

the instruction chain for multi-GPU programming can be extended as:

1. create streams for all GPUs

2. allocate host memory
3. allocate device memory for all GPUs
4. split work into chunks
5. transfer data in chunks to all devices / streams
6. launch kernels per device / stream
7. synchronize host and all devices / streams
8. transfer data from device
9. destroy streams
10. free all memory

A full example of multi-GPU programming can be found below. As it is not directly part of this course, we will not go into too much detail of the implementation. In this example, data is ”encrypted” by a CPU function and ”decrypted” on GPU. The most interessting part is in `mgpu.cu`, the CUDA file that contains the source code for splitting data and launching the kernels.

encryption.cuh

```
#pragma once

#include <cstdint>

__host__ __device__ __forceinline__
uint32_t hash32(uint32_t x) {

        x ^= x >> 16;
        x *= 0x85ebca6bU;
        x ^= x >> 13;
        x *= 0xc2b2ae35U;
        x ^= x >> 16;

        return x;
}

__host__ __device__ __forceinline__
uint64_t permute64(uint64_t x, uint64_t num_iters) {

    constexpr uint64_t mask = (1ULL << 32) - 1;

    for (uint64_t iter = 0; iter < num_iters; ++iter) {
        const uint64_t upper = x >> 32;
        const uint64_t lower = x & mask;
        const uint64_t mixer = hash32(static_cast<uint32_t>(upper));
        const uint64_t lower_mixed = (lower ^ mixer) & mask;
```

```
        x = upper | (lower_mixed << 32);
    }

    return x;
}

__host__ __device__ __forceinline__
uint64_t unpermute64(uint64_t x, uint64_t num_iters) {

    constexpr uint64_t mask = (1ULL << 32) - 1;

    for (uint64_t iter = 0; iter < num_iters; iter++) {
        const uint64_t upper = x & mask;
        const uint64_t lower = x >> 32;
        const uint64_t mixer = hash32(static_cast<uint32_t>(upper));
        const uint64_t lower_mixed = (lower ^ mixer) & mask;

        x = (upper << 32) | lower_mixed;
    }

    return x;
}
```

helpers.cuh

```
#pragma once

#include <cuda_runtime.h>

#include <iostream>
#include <fstream>
#include <filesystem>
#include <cstdint>
#include <string>
#include <cassert>

inline uint64_t sdiv(uint64_t a, uint64_t b) {
    assert(b && "[Error] (sdiv): division by zero");
    return (a + b - 1) / b;
}

inline void check_last_error ( ) {
    cudaError_t err;
    if ((err = cudaGetLastError()) != cudaSuccess) {
        std::cout << "CUDA error: " << cudaGetErrorString(err) << " : "
                  << __FILE__ << ", line " << __LINE__ << std::endl;
            std::terminate();
```

```
    }
}

inline bool file_exists(const char * encrypted_file) {
    return (
        std::filesystem::exists(encrypted_file)
        and std::ifstream(encrypted_file, std::ios::binary).good()
    );
}

inline void write_encrypted_to_file(const char* encrypted_file, uint64_t *
↪   data, size_t size) {
    std::ofstream encrypted(encrypted_file, std::ios::binary);
    if (not encrypted) throw std::runtime_error(
        "[Error] (write_encrypted_to_file): Couldn't open file: "
        + std::string{encrypted_file}
    );
    encrypted.write(reinterpret_cast<char*>(data), size);
    if (not encrypted) throw std::runtime_error(
        "[Error] (write_encrypted_to_file): Writing failed for file: "
        + std::string{encrypted_file}
    );
}

inline void read_encrypted_from_file(const char* encrypted_file, uint64_t *
↪   data, size_t size) {
    std::ifstream encrypted(encrypted_file, std::ios::binary);
    if (not encrypted) throw std::runtime_error(
        "[Error] (read_encrypted_from_file): Couldn't open file: "
        + std::string{encrypted_file}
    );
    encrypted.read(reinterpret_cast<char*>(data), size);
    if (not encrypted) throw std::runtime_error(
        "[Error] (read_encrypted_from_file): Reading failed for file: "
        + std::string{encrypted_file}
    );
}

class Timer {

    float time;
    const uint64_t gpu;
    cudaEvent_t ying, yang;

public:

    Timer (uint64_t gpu=0) : gpu(gpu) {
        cudaSetDevice(gpu);
```

```
        cudaEventCreate(&ying);
        cudaEventCreate(&yang);
    }

    ~Timer ( ) {
        cudaSetDevice(gpu);
        cudaEventDestroy(ying);
        cudaEventDestroy(yang);
    }

    void start ( ) {
        cudaSetDevice(gpu);
        cudaEventRecord(ying, 0);
    }

    void stop (std::string label) {
        cudaSetDevice(gpu);
        cudaEventRecord(yang, 0);
        cudaEventSynchronize(yang);
        cudaEventElapsedTime(&time, ying, yang);
        std::cout << "TIMING: " << time << " ms (" << label << ")" <<
        ↪  std::endl;
    }
};

```

mgpu.cu

```
#include "helpers.cuh"
#include "encryption.cuh"

#include <omp.h>

#include <cstdint>
#include <iostream>
#include <algorithm>

void encrypt_cpu(uint64_t * data, uint64_t num_entries,
                 uint64_t num_iters, bool parallel=true) {

    #pragma omp parallel for if (parallel)
    for (uint64_t entry = 0; entry < num_entries; ++entry)
        data[entry] = permute64(entry, num_iters);
}

__global__
void decrypt_gpu(uint64_t * data, uint64_t num_entries,
                 uint64_t num_iters) {
```

```
21
22      const uint64_t thrdID = blockIdx.x*blockDim.x+threadIdx.x;
23      const uint64_t stride = blockDim.x*gridDim.x;
24
25      for (uint64_t entry = thrdID; entry < num_entries; entry += stride)
26          data[entry] = unpermute64(data[entry], num_iters);
27  }
28
29  bool check_result_cpu(uint64_t * data, uint64_t num_entries,
30                        bool parallel=true) {
31
32      bool match = true;
33      #pragma omp parallel for shared(match) schedule(dynamic) if(parallel)
34      for (uint64_t entry = 0; entry < num_entries; ++entry) {
35          bool ok;
36          #pragma omp atomic read
37          ok = match;
38          if (not ok) continue;
39          if (data[entry] != entry) {
40              #pragma omp atomic write
41              match = false;
42          }
43      }
44
45      return match;
46  }
47
48  int main (int argc, char* argv[]) {
49
50      const char * encrypted_file = "/dli/task/encrypted";
51
52      Timer timer;
53
54      const uint64_t num_entries = 1UL << 26;
55      const uint64_t num_iters = 1UL << 10;
56      const bool openmp = true;
57
58      const uint64_t num_gpus = 4;
59      const uint64_t num_streams = 32;
60
61      // 2D array containing number of streams for each GPU.
62      cudaStream_t streams[num_gpus][num_streams];
63
64      // For each available GPU device...
65      for (uint64_t gpu = 0; gpu < num_gpus; ++gpu) {
66          // ...set as active device...
67          cudaSetDevice(gpu);
68          for (uint64_t stream = 0; stream < num_streams; stream++)
```

```
69              // ...create and store its number of streams.
70              cudaStreamCreate(&streams[gpu][stream]);
71      }
72      check_last_error();
73
74      // Each stream needs num_entries/num_gpus/num_streams data.
75      // Because of ceil_division in sdiv function, we may have
76      // extra allocation in the last gpu which should be handled
77      // specially when allocating the chunks per gpu and when
78      // chunking each gpu data across its streams
79      const uint64_t gpu_size = sdiv(num_entries, num_gpus);
80
81      // Store GPU data pointers in an array.
82      uint64_t* data_cpu;
83      uint64_t* data_gpu[num_gpus];
84      cudaMallocHost(&data_cpu, sizeof(uint64_t)*num_entries);
85
86      // For each gpu device...
87      for (uint64_t gpu = 0; gpu < num_gpus; ++gpu) {
88
89          // ...set device as active...
90          cudaSetDevice(gpu);
91
92          // ...use a GPU chunk's worth of data to calculate indices and width...
93          const uint64_t lower = gpu_size*gpu;
94          const uint64_t upper = min(lower + gpu_size, num_entries); // #1 handle
            ↪  chunk size from ceil division
95          const uint64_t width = upper-lower;
96
97          // ...allocate memory.
98          cudaMalloc(&data_gpu[gpu], sizeof(uint64_t)*width);
99      }
100     check_last_error();
101
102     if (not file_exists(encrypted_file)) {
103         encrypt_cpu(data_cpu, num_entries, num_iters, openmp);
104         write_encrypted_to_file(encrypted_file, data_cpu,
            ↪  sizeof(uint64_t)*num_entries);
105     } else {
106         read_encrypted_from_file(encrypted_file, data_cpu,
            ↪  sizeof(uint64_t)*num_entries);
107     }
108
109     timer.start();
110
111     const uint64_t common_stream_size = sdiv(gpu_size, num_streams);
112     const uint64_t last_stream_size = sdiv(num_entries - ((num_gpus -1) *
        ↪  gpu_size), num_streams);  // #2 handle chunk size from ceil division
```

```
113
114     // For each gpu...
115     for (uint64_t gpu = 0; gpu < num_gpus; ++gpu) {
116         // ...set device as active.
117         cudaSetDevice(gpu);
118         const uint64_t data_gpu_start = gpu * gpu_size;
119         const uint64_t data_gpu_end = std::min(data_gpu_start + gpu_size,
            ↪  num_entries); // #3 handle chunk size from ceil division
120         const uint64_t data_gpu_size = data_gpu_end - data_gpu_start;
121
122         const uint64_t stream_size = (gpu != (num_gpus - 1)) ?
            ↪  common_stream_size : last_stream_size;
123         // For each stream (on each GPU)...
124         for (uint64_t stream = 0; stream < num_streams; ++stream) {
125
126             // Calculate index offset for this stream's chunk of data within
                ↪  the GPU's chunk of data...
127             const uint64_t data_stream_start = stream_size * stream;
128             const uint64_t data_stream_end = std::min(data_stream_start +
                ↪  stream_size, data_gpu_size);  // #4 handle chunk size from ceil
                ↪  division
129             const uint64_t data_stream_size = data_stream_end -
                ↪  data_stream_start;
130             if (data_stream_size == 0) break;
131
132             // ...perform async HtoD memory copy...
133             cudaMemcpyAsync(data_gpu[gpu] + data_stream_start, // This stream's
                ↪  data within this GPU's data.
134                             data_cpu + data_gpu_start + data_stream_start,
                                ↪  // This stream's data within all CPU data.
135                             sizeof(uint64_t) * data_stream_size,      // This
                                ↪  stream's chunk size worth of data.
136                             cudaMemcpyHostToDevice,
137                             streams[gpu][stream]);       // Using this stream
                                ↪  for this GPU.
138
139             decrypt_gpu<<<80*32, 64, 0, streams[gpu][stream]>>>    // Using
                ↪  this stream for this GPU.
140                 (data_gpu[gpu] + data_stream_start,                    //
                    ↪  This stream's data within this GPU's data.
141                  data_stream_size,
                     ↪  // This stream's chunk size worth of data.
142                  num_iters);
143
144             cudaMemcpyAsync(data_cpu + data_gpu_start + data_stream_start,
                ↪  // This stream's data within all CPU data.
145                             data_gpu[gpu] + data_stream_start, // This stream's
                                ↪  data within this GPU's data.
```

```
                            sizeof(uint64_t) * data_stream_size,
                            cudaMemcpyDeviceToHost,
                            streams[gpu][stream]);       // Using this stream
                            ↪  for this GPU.
        }
    }

    // Synchronize streams to block on memory transfer before checking on host.
    for (uint64_t gpu = 0; gpu < num_gpus; ++gpu) {
        cudaSetDevice(gpu);
        for (uint64_t stream = 0; stream < num_streams; stream++) {
            cudaStreamSynchronize(streams[gpu][stream]);
        }
    }

    timer.stop("total time on GPU");
    check_last_error();

    const bool success = check_result_cpu(data_cpu, num_entries, openmp);
    std::cout << "STATUS: test "
              << ( success ? "passed" : "failed")
              << std::endl;

    for (uint64_t gpu = 0; gpu < num_gpus; ++gpu) {
        cudaSetDevice(gpu);
        for (uint64_t stream = 0; stream < num_streams; stream++) {
            cudaStreamDestroy(streams[gpu][stream]);
        }
    }
    check_last_error();

    // deallocate memory
    cudaFreeHost(data_cpu);
    for (uint64_t gpu = 0; gpu < num_gpus; ++gpu) {
        cudaSetDevice(gpu);
        cudaFree(data_gpu[gpu]);
    }
    check_last_error();
}
```

8.8 Optimization Tips

The number of GPUs, streams, blocks, and threads, can have a huge impact on the runtime performance of an application.

Some rules of thumb that can be used to start with:

- Number of blocks: Launch enough blocks such that all SMs are fully used. Launch at least as many blocks as SMs, often a multiple of SM to keep all SMs busy while others wait (hide memory latency). For example, if you have 22 SMs, launching `n_blocks = 32 * n_SMs` is a solid choice to start with.
- Number of threads: Make the number of threads per block a multiple of warp size (usually 32), e.g. 128, 256, 512. This ensures full utilization of warps and avoids partial warps (which are inefficient).

The compute capabilities can give a hint about features that are available on the specific architecture of the GPU.

8.9 Common Mistakes

Additionally, there are very common mistakes that that should be checked regularly.

- Forgetting `cudaDeviceSynchronize()`
- Not checking for errors
- Array out-of-bounds
- Not copying memory properly
- Mixing host/device pointers
- Not using pinned memory for async copies